Computer Vision
Models, Learning and Inference

Algorithms Implementations in MATLAB

Student: Stefan Stavrev

Supervisor: Dr. Simon J. D. Prince

Devoted to God and Science

Chapter 4

Fitting Probability Models

4.1 Maximum likelihood learning of univariate normal distribution

```
function [mu, var] = mle_norm (x)
    I = length(x);
    mu = sum(x) / I;
    var = sum((x - mu) .^ 2) / I;
end
```

Example 4.1

```
% Generate random 100 values from the normal distribution with
% mean value original_mu and standard deviation original_sig.
original_mu = 5;
original_sig = 8;
r = original_mu + original_sig .* randn(100,1);

% Estimate the mean and the variance for the data in r.
[estimated_mu, estimated_var] = mle_norm(r);
estimated_sig = sqrt(estimated_var);

% Estimate and print the error for the mean and the standard deviation.
muError = abs(original_mu - estimated_mu);
sigError = abs(original_sig - estimated_sig);
disp([muError, sigError]);

% Plot the original and the estimated models for comparison.
x = -20:0.01:30;
original = normpdf(x, original_mu, original_sig);
estimated = normpdf(x, estimated_mu, estimated_sig);
plot(x, original, 'g', x, estimated, 'b');
```

The resulting plot (Figure 4.1) shows that the difference between the original and the estimated distribution is small.

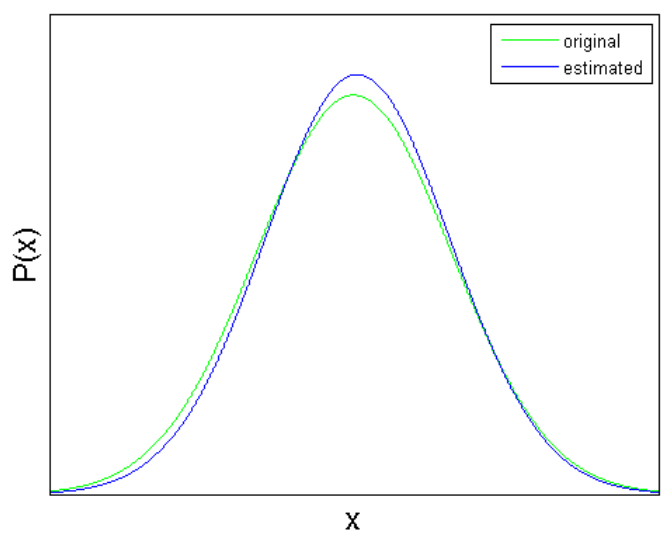


Figure 4.1 The original (green) and the estimated (blue) distributions.

4.2 MAP learning of univariate normal distribution

```
function [mu, var] = map_norm (x, alpha, beta, gamma, delta)
    I = length(x);
    mu = (sum(x) + gamma*delta) / (I + gamma);
    var_up = sum((x - mu) .^ 2) + 2*beta + gamma*(delta-mu)^2;
    var_down = I + 3 + 2*alpha;
    var = var_up / var_down;
end
```

Example 4.2

The code is in the script file: “Chapter 4/4.2 MAP for Univariate Normal Distribution/map_norm_ex1.m”. It is not shown here because it is very similar to (Example 4.1). There is one additional variable I, which specifies the number of data points. The example code was executed four times with different values for the variable I: 5, 30, 1000, 1000000, and the resulting plots are shown in (Figure 4.2). It can be seen that as the number of data points increases, the MAP estimate is very close to the original distribution (and the ML estimate).

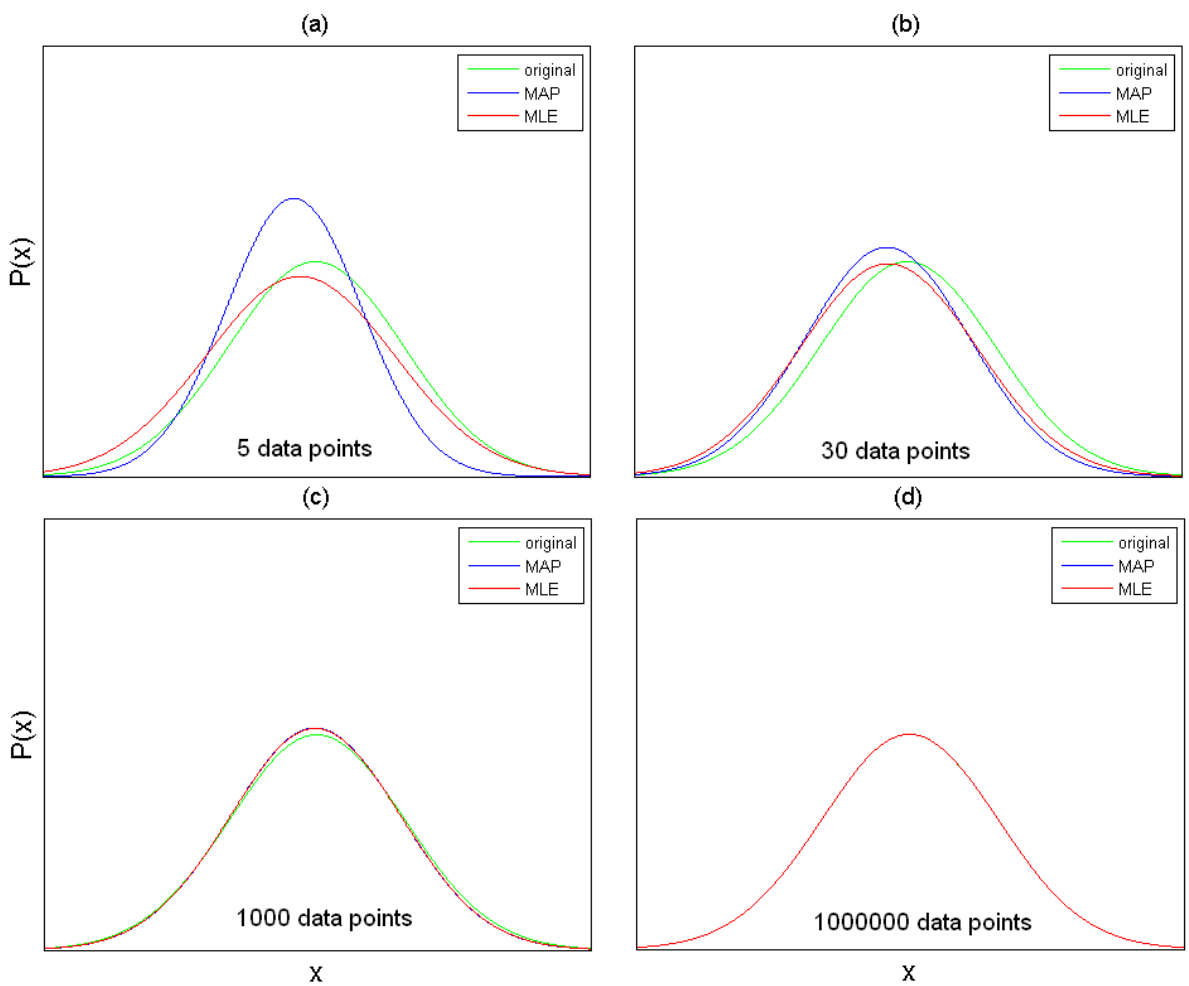


Figure 4.2 The number of data points was changed to (a) 5, (b) 30, (c) 1000, (d) 1000000. As the number of data points increases, the MAP estimate is getting closer to the original distribution and the MLE estimate.

This can also be confirmed by the printed output from (Example 4.2). As the number of data points increases, the MAP estimation of the mean is getting closer to the original mean which was set to 5, and the errors for the estimated mean and variance are getting smaller.

```
% I = 5
Estimated mean: 2.9894
Errors: 2.0106, 1.8206

% I = 30
Estimated mean: 3.1481
Errors: 1.8519, 0.49785

% I = 1000
Estimated mean: 4.8456
Errors: 0.15441, 0.24591

% I = 1000000
Estimated mean: 4.9994
Errors: 0.00055452, 0.00081419
```

4.3 Bayesian approach to univariate normal distribution

```
function [alpha_post, beta_post, gamma_post, delta_post, x_prediction]...
    = by_norm(x, alpha_prior, beta_prior, gamma_prior, delta_prior, x_test)
    % Compute posterior parameters.
    I = length(x);
    alpha_post = alpha_prior + I/2;
    beta_post= sum(x.^2)/2 + beta_prior + (gamma_prior*delta_prior^2)/2 ...
         - (gamma_prior*delta_prior + sum(x))^2 / (2*(gamma_prior + I));
    gamma_post = gamma_prior + I;
    delta_post = (gamma_prior*delta_prior + sum(x)) / (gamma_prior + I);

    % Compute intermediate parameters.
    alpha_int = alpha_post + 0.5;
    beta_int= (x_test.^2)/2 + beta_post + (gamma_post*delta_post^2)/2 - ...
        (gamma_post*delta_post + x_test).^2 / (2*gamma_post + 2);
    gamma_int = gamma_post + 1;

    % Predict values for x_test.
    temp1 = sqrt(gamma_post) * (beta_post^alpha_post) * gamma(alpha_int);
    x_prediction_up = repmat(temp1, 1, length(x_test));
    x_prediction_down = sqrt(2*pi) * sqrt(gamma_int) * gamma(alpha_post)...
        * beta_int.^alpha_int;
    x_prediction =  x_prediction_up ./ x_prediction_down;

end
```

Example 4.3

The example code is in the script file: “Chapter 4/4.3 Bayesian Approach to Univariate Normal Distribution/ by_norm_ex1.m”. It is very similar to (Example 4.1) and (Example 4.2). In (Figure 4.3), the

two plots in the first row are with linear scale on both axis, while the two plots in the second row are with linear scale on the x-axis and a logarithmic scale on the y-axis. As the number of data points decreases, the Bayesian estimate becomes less certain, while the MAP estimate is overconfident. Visually this means that the Bayesian estimate is wider than the MAP estimate. As the number of data points increases, they both get closer to the original distribution.

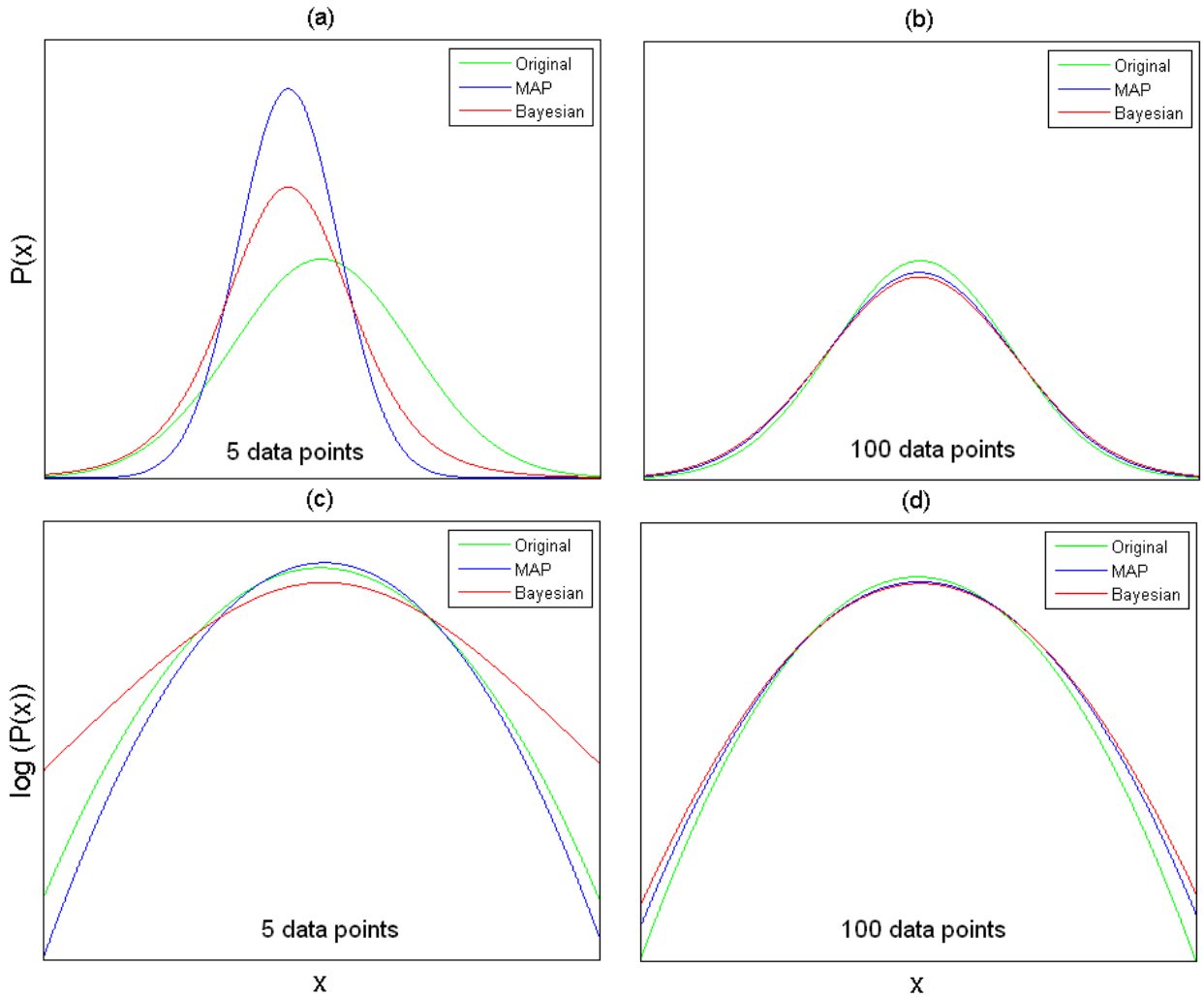


Figure 4.3 For the linear scale plots in (a) and (b) the number of data points was set to 5 and 100 respectively. The same changes were made for the plots in (c) and (d), which have logarithmic scale on the y-axis. It can be seen that the Bayesian estimate is wider than the MAP esitmate in all cases. We can say that the Bayesian estimate is "less certain", while the MAP estimate is "overconfident". This difference is significant for small number of data points, but as the number of data points increases both solutions converge toward the original distribution.

4.4 Maximum likelihood learning of categorical parameters

```
function [theta] = mle_cat (x, K)
    [counts] = hist(x, 1:K);
    theta = counts ./ sum(counts);
end
```

Example 4.4

```
% Generate random values from the categorical distribution
% with 6 categories and the corresponding probabilities.
original_probabilities = [0.25,0.15,0.1,0.1,0.15,0.25];
r1 = randsample (6, 200, true, original_probabilities);
r2 = randsample (6, 20000, true, original_probabilities);

% Estimate the categorical distribution parameters from the data.
estimated_probabilities1 = mle_cat (r1, 6);
estimated_probabilities2 = mle_cat (r2, 6);

% Plot the original and the estimated models for comparison.
...
```

The complete example code can be found in "Chapter 4/4.4 MLE for Categorical Distribution /mle_cat_ex1.m". The resulting plots are shown in (Figure 4.4).

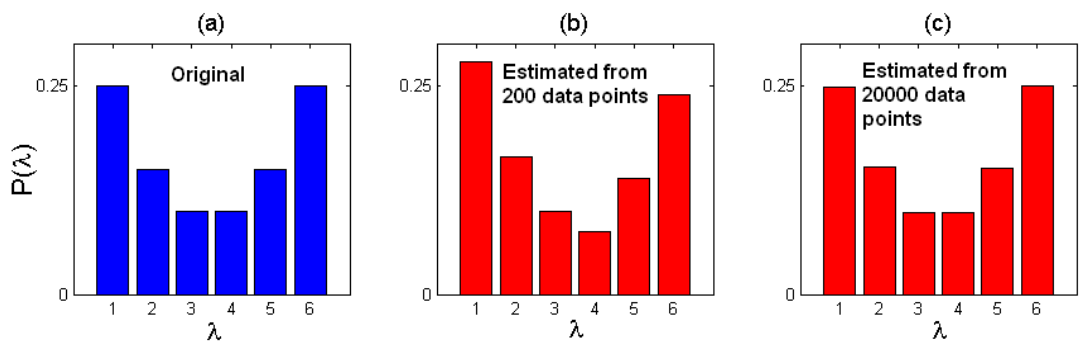


Figure 4.4 The original categorical distribution (a) is shown in blue, while the estimated categorical distributions (b) and (c) are shown in red. As the number of data points increases (left to right) the estimated distribution gets closer to the original distribution.

4.5 MAP learning of categorical parameters

Before we look at MAP learning of categorical parameters we will explore the Dirichlet distribution. The Dirichlet distribution is conjugate to the categorical distribution, so it can be used to model the prior over the categorical parameters. It is hard to visualize the Dirichlet distribution, but we can draw samples and plot them, and then try to detect relations between the distribution parameters and the samples.

For this purpose, a simple function is presented for sampling the Dirichlet distribution [2], followed by an example. First, the function draws samples from independent gamma distributions with common parameters, and then they are normalized. The function returns a matrix of size NxK, where N is the number of generated samples and K is the number of Dirichlet parameters. Each row is one sample from the Dirichlet distribution.

Dirichlet distribution sampling

```
% Input:        alpha - Dirichlet distribution parameters,
%               N - number of samples to generate.
% Output:       r - random samples from the Dirichlet distribution.
```

```
function r = randdir (alpha, N)
    K = length(alpha);
    r = gamrnd(repmat(alpha,N,1),1,N,K);
    r = r ./ repmat(sum(r,2),1,K);
end
```

Example 4.5

```
% Generate random values from the Dirichlet distribution with 6 parameters.
%r = randdir ([1 1 1 1 1 1], 3);
%r = randdir ([1000 1000 1000 1000 1000 1000], 3);
r = randdir ([10 100 1000 1000 100 10], 3);

% Plot the first sample.
subplot(1,3,1);
bar(r(1,:), 'b');
axis([0,7,0,0.6]);
xlabel('x', 'FontSize', 16);
ylabel('P(x)', 'FontSize', 16);
set(gca,'YTick',[0, 0.25]);
title('(g)', 'FontSize', 14);

% Plot the second sample.
subplot(1,3,2);
bar(r(2,:), 'b');
axis([0,7,0,0.6]);
xlabel('x', 'FontSize', 16);
ylabel('', 'FontSize', 16);
set(gca,'YTick',[0, 0.25]);
title('(h)', 'FontSize', 14);

% Plot the third sample.
subplot(1,3,3);
bar(r(3,:), 'b');
axis([0,7,0,0.6]);
xlabel('x', 'FontSize', 16);
ylabel('', 'FontSize', 16);
set(gca,'YTick',[0, 0.25]);
title('(i)', 'FontSize', 14);
```

For this example a Dirichlet distribution with six parameters is used. The resulting plots are shown in (Figure 4.5). In each row there are three samples, each represented by one plot. For each row, different Dirichlet parameters were used: [1 1 1 1 1 1] for the first row (red bars), [1000 1000 1000 1000 1000 1000] for the second row (green bars), and [10 100 1000 1000 100 10] for the third row (blue bars). As already explained in “3.4 Dirichlet Distribution” [1], the relative values of the parameters determine the expected value for the distribution, and the absolute values determine the concentration around the expected value.

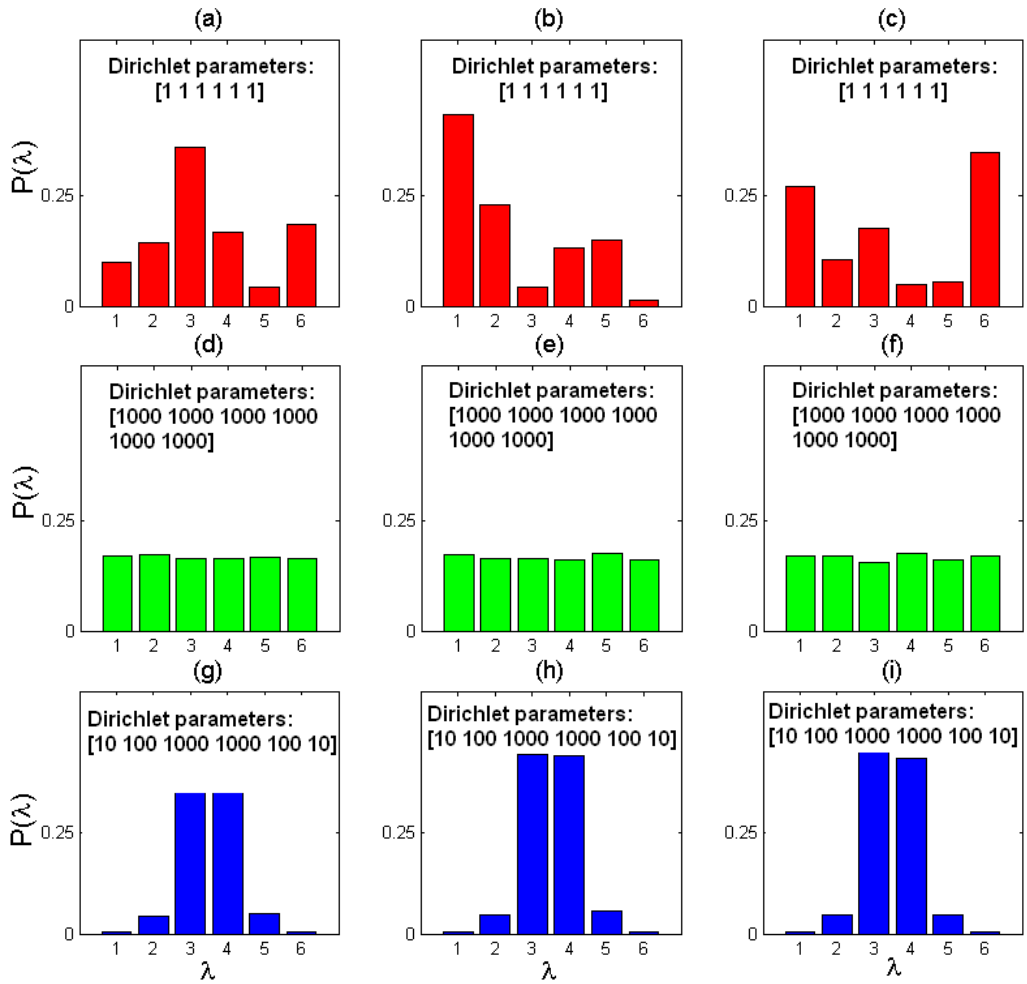


Figure 4.5 For each row, different Dirichlet parameters were used: [1 1 1 1 1 1] in (a-c), [1000 1000 1000 1000 1000 1000] in (d-f), and [10 100 1000 1000 100 10] in (g-i). Notice the relation between the Dirichlet parameters and the resulting categorical distributions.

Now that we have explored the Dirichlet distribution, we can continue talking about MAP learning of categorical parameters. The MAP solution using prior with all parameters set to 1 is exactly the same as the MLE solution (Figure 4.6). In (Figure 4.7) different prior is used, and for small amount of data points the MAP solution is closer to the prior, while for large amount of data the MAP solution gets closer to the original distribution, and of course the MLE estimate.

MAP learning of categorical distribution

```
% Description: MAP learning of categorical distribution.
% Input: x - a row or a column vector,
%        alpha - Dirichlet prior.
% Output: theta - K categorical distribution parameters.
function [theta] = map_cat (x, alpha)
    I = length(x);
    K = length(alpha);
    [counts] = hist (x, 1:K);
    theta = (counts - 1 + alpha) ./ (I - K + sum(alpha));
end
```

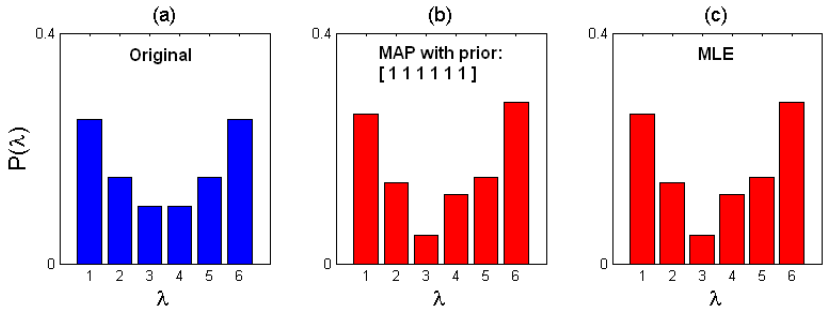


Figure 4.6 The original categorical distribution is shown in blue (a), while the estimated distributions (MAP and MLE) are shown in red (b-c). Because all the parameters for the MAP prior are set to 1 (b), the prior is uniform and does not carry any useful information, therefore the MAP estimate (b) is exactly the same as the MLE estimate (c). The code that produced this plot can be found in "Chapter 4/4.5 MAP for Categorical Distribution/map_cat_ex1.m".

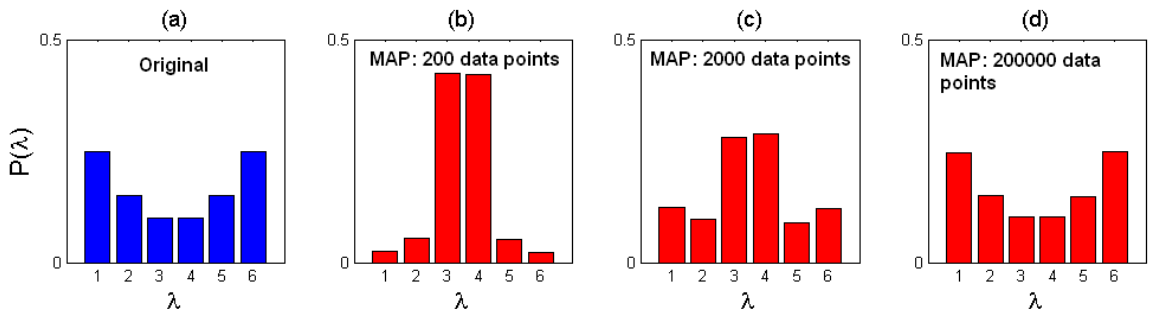


Figure 4.7 The original categorical distribution is shown in blue (a), while the MAP estimated distributions are shown in red (b-d). The prior used here is [10 100 1000 1000 100 10], the same as in (Figure 5.1 g-i). For small amount of data points the MAP estimate (b) looks very similar to the prior (see Figure 5.1 g-i). But as the number of data points increases (b-d) the MAP estimate gets closer to the original distribution (d). The code that produced this plot can be found in "Chapter 4/4.5 MAP for Categorical Distribution/map_cat_ex2.m".

4.6 Bayesian approach to categorical distribution

```
function [alpha_post, prediction] = by_cat (x, alpha_prior)
    % Compute posterior.
    K = length (alpha_prior);
    [counts] = hist (x, 1:K);
    alpha_post = alpha_prior + counts;

    % Predict.
    prediction = alpha_post ./ sum(alpha_post);
end
```

The example code can be found in "Chapter 4/4.6 Bayesian Approach to Categorical Distribution /by_cat_ex1.m", and the resulting plot is shown in (Figure 4.8). One important thing to understand is that the Bayesian solution is less confident than the MAP solution. This can be seen in (Figure 4.8 b-c) where the MAP solution assigns 0 probability to the value 3 which was not observed in the data, while the Bayesian aproach assigns it a small probability. In (Figure 4.8 d-e) we can see that for large amount of data, both the MAP and the Bayesian solution are close to the original distribution.

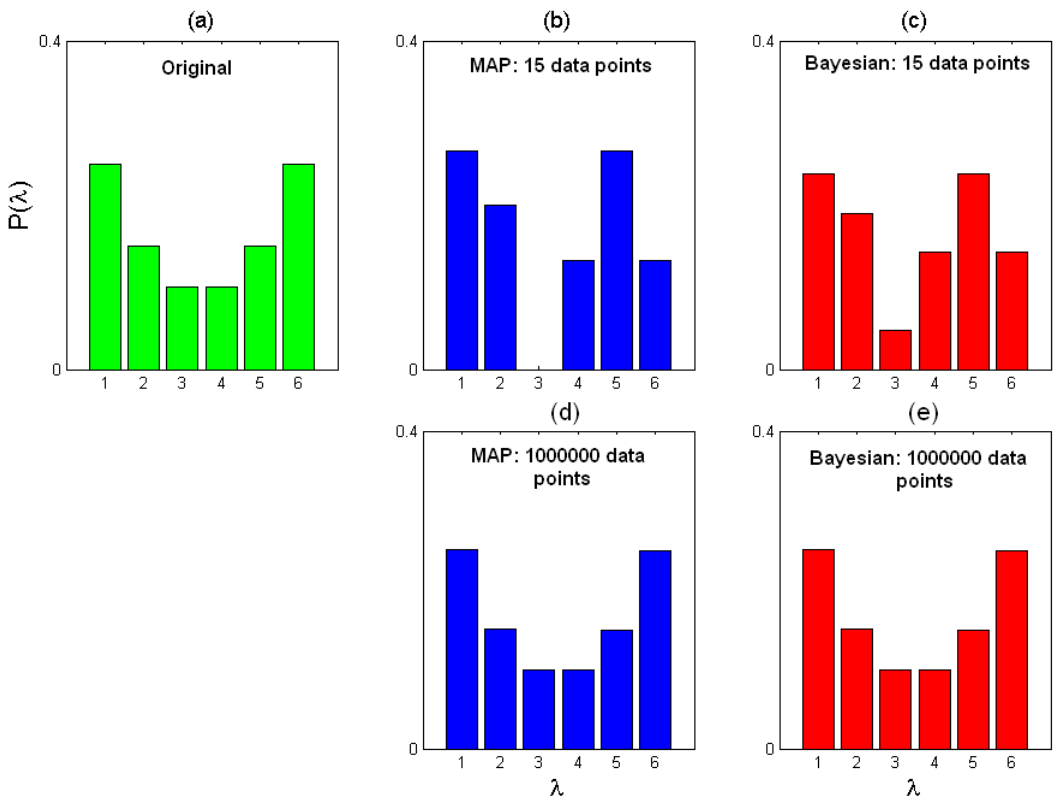


Figure 4.8 The original distribution is shown in (a). The MAP solution in (b) assigns 0 probability to the value 3 which was not observed in the dataset, while the Bayesian solution (c) assigns it a small probability. For large amount of data, both the MAP and the Bayesian solution are close to the original distribution.

Chapter 6

Learning and Inference in Vision

6.1 Basic generative classifier

```
% Description: Classification based on multivariate measurement vector.
% Input: x_train - matrix of size (#training examples X dimensionality+1).
%                  The number of columns is the dimensionality of the data
%                  plus 1 for the last column, which contains class
%                  assignments for the corresponding row vectors.
%        x_test  - matrix of size (#test examples X dimensionality). The
%                  number of columns is one less than the number of columns
%                  in x_train, because there is no column for class
%                  assignments.
%        K       - number of classes. The classes throughout the code
%                  are assumed to be represented by integers 1:K.
% Output: lambda - categorical prior with K parameters, one for each
%                  of the K classes/categories.
%         mu     - matrix of size (K X dimensionality). Each row indexed by
%                  k, is the mean of the training examples in class k.
%         sig    - cell of K covariance matrices, where sig{k} is the
%                  covariance matrix for the class k.
%         posterior - matrix of size (size(x_test,1) X K), where each
%                  row is a posterior probability distribution over the
%                  K classes, for the corresponding row vector from the
%                  test data.
function [lambda, mu, sig, posterior] = class_gen (x_train, x_test, K)
    % Separate the training data based on the values in the last column,
    % which represents the class. The result is one matrix of training
    % data, per class. Some matrices may be empty, if the corresponding
    % class was not observed in the data. At the same time, compute
    % the counts for each class, that is how  many training examples belong
    % to each class.
    I = size (x_train, 1);
    dimensionality = size (x_train, 2) - 1;
    x_train_per_class = cell (1, K);
    class_counts = zeros (1, K);
    for i = 1 : I
        k = x_train (i, end);
        current_row = x_train (i, :);
        % Remove the last column.
        current_row (end) = [];
        x_train_per_class{k} = [x_train_per_class{k}; current_row];
        % Increase class count.
        class_counts(k) = class_counts(k) + 1;
    end

    % Compute mu, sigma and lambda for each class.
    mu = zeros (K, dimensionality);
    sig = cell (1, K);
    lambda = zeros (K, 1);
    for k = 1 : K
        % Compute mu.
        mu(k,:) = sum(x_train_per_class{k},1) ./ class_counts(k);

        % Compute sigma.
        sig{k} = zeros (dimensionality, dimensionality);
        for i = 1 : class_counts(k)
```

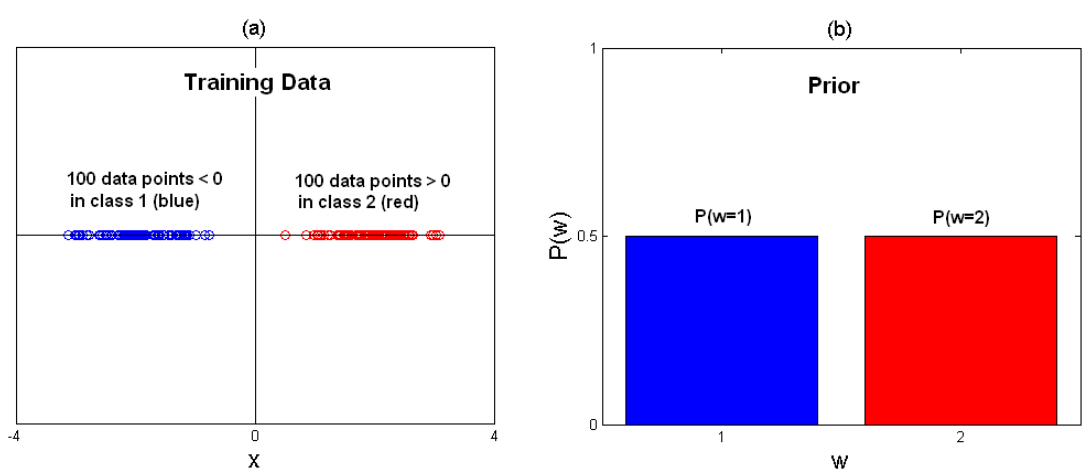
```
            mat = x_train_per_class{k}(i,:) - mu(k,:);
            mat = mat' * mat;
            sig{k} = sig{k} + mat;
        end
        sig{k} = sig{k} ./ class_counts(k);

        % Compute lambda.
        lambda(k) = class_counts(k) / I;
    end

    % Compute likelihoods for each class for the test data.
    likelihoods = zeros (size(x_test,1), K);
    for k = 1 : K
        likelihoods(:,k) = mvnpdf (x_test, mu(k,:), sig{k});
    end

    % Classify the data with Bayes' rule.
    denominator = 1 ./ (likelihoods * lambda);
    posterior = (likelihoods * diag(lambda));
    posterior = (posterior' * diag(denominator))';
end
```

The example code can be found in “Chapter 6/6.1 Basic Generative Classifier/class_gen_ex1.m”. The data in this example is one-dimensional, however the code is general and works for multivariate measurement vectors also. The resulting plots are shown in (Figure 6.1). Training data was generated from two normal distributions with mean values -2 and 2, and standard deviation value 0.5 for both (Figure 6.1 (a)). Negative values were assigned to class 1 (blue), while the positive values were assigned to class 2 (red). Both classes contain equal number of data points 100, therefore the prior probabilities $P(w = 1)$ and $P(w = 2)$ are equal (Figure 6.1 (b)). The result from the likelihood computations (Figure 6.1 (c)) agrees with the way the training data was generated, that is by using two normal distributions with mean values -2 and 2, and standard deviation value 0.5 for both. Finally, the posterior was computed with Bayes’ rule (Figure 6.1 (d)). The result agrees with our expectations. Given that class 1 contains only negative values and class 2 contains positive values, we would expect that the probability for a positive value to be classified in class 2, should be higher than the probability for it to be classified in class 1. The same holds for a negative value, we would expect it to be classified in class 1, rather than in class 2.



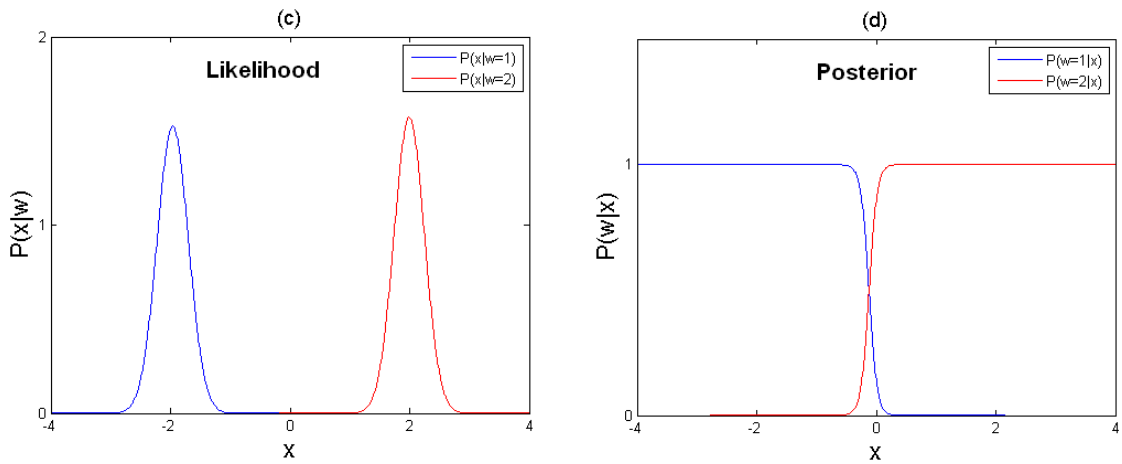


Figure 6.1 (a) The training data was generated from two normal distributions with mean values -2 and 2, and standard deviation value 0.5 for both. Negative values were assigned to class 1 (blue), while the positive values were assigned to class 2 (red). (b) Both classes contain equal number of data points 100, therefore the prior probabilities $P(w = 1)$ and $P(w = 2)$ are equal. (c) The result from the likelihood computations agrees with the way the training data was generated, that is by using two normal distributions with mean values -2 and 2, and standard deviation value 0.5 for both. (d) Finally, the posterior was computed with Bayes' rule.

Chapter 7

Modeling Complex Data Densities

7.1 Fitting mixture of Gaussians

```
% Description: Fitting mixture of Gaussians.
% Input: x       - each row is one datapoint.
%        K       - number of Gaussians in the mixture.
%        precision - the algorithm stops when the difference between
%                    the previous and the new likelihood is < precision.
%                    Typically this is a small number like 0.01.
% Output:
%        lambda  - lambda(k) is the weight for the k-th Gaussian.
%        mu      - mu(k,:) is the mean for the k-th Gaussian.
%        sig     - sig{k} is the covariance matrix for the k-th Gaussian.
function [lambda, mu, sig] = fit_mog (x, K, precision)
    % Initialize all values in lambda to 1/K.
    lambda = repmat (1/K, K, 1);

    % Initialize the values in mu to K randomly chosen unique datapoints.
    I = size (x, 1);
    K_random_unique_integers = randperm(I);
    K_random_unique_integers = K_random_unique_integers(1:K);
    mu = x (K_random_unique_integers,:);

    % Initialize the variances in sig to the variance of the dataset.
    sig = cell (1, K);
    dimensionality = size (x, 2);
    dataset_mean = sum(x,1) ./ I;
    dataset_variance = zeros (dimensionality, dimensionality);
    for i = 1 : I
        mat = x (i,:) - dataset_mean;
        mat = mat' * mat;
        dataset_variance = dataset_variance + mat;
    end
    dataset_variance = dataset_variance ./ I;
    for i = 1 : K
        sig{i} = dataset_variance;
    end

    % The main loop.
    iterations = 0;
    previous_L = 1000000; % just a random initialization
    while true
        % Expectation step.
        l = zeros (I,K);
        r = zeros (I,K);
        % Compute the numerator of Bayes' rule.
        for k = 1 : K
            l(:,k) = lambda(k) * mvnpdf (x, mu(k,:), sig{k});
        end

        % Compute the responsibilities by normalizing.
        s = sum(l,2);
        for i = 1 : I
            r(i,:) = l(i,:) ./ s(i);
        end
```

```
        % Maximization step.
        r_summed_rows = sum (r,1);
        r_summed_all = sum(sum(r,1),2);
        for k = 1 : K
            % Update lambda.
            lambda(k) = r_summed_rows(k) / r_summed_all;

            % Update mu.
            new_mu = zeros (1,dimensionality);
            for i = 1 : I
                new_mu = new_mu + r(i,k)*x(i,:);
            end
            mu(k,:) = new_mu ./ r_summed_rows(k);

            % Update sigma.
            new_sigma = zeros (dimensionality,dimensionality);
            for i = 1 : I
                mat = x(i,:) - mu(k,:);
                mat = r(i,k) * (mat' * mat);
                new_sigma = new_sigma + mat;
            end
            sig{k} = new_sigma ./ r_summed_rows(k);
        end

        % Compute the log likelihood L.
        temp = zeros (I,K);
        for k = 1 : K
            temp(:,k) = lambda(k) * mvnpdf (x, mu(k,:), sig{k});
        end
        temp = sum(temp,2);
        temp = log(temp);
        L = sum(temp);
        %disp(L);

        iterations = iterations + 1;
        %disp([num2str(iterations) ': ' num2str(L)]);
        if abs(L - previous_L) < precision
            msg= [num2str(iterations) ' iterations, log-likelihood = ', ...
                num2str(L)];
            disp(msg);
            break;
        end

        previous_L = L;
    end
end
```

The initialization used here is as suggested in the Algorithms Booklet that accompanies Dr. Prince's book [1]. All the weights are initialized as $\lambda_i = 1/K$, where K is the number of components in the Gaussian mixture. The means $\boldsymbol{\mu}_i$ are initialized to K randomly chosen unique datapoints, and the covariances $\boldsymbol{\Sigma}_i$ to the covariance of the whole dataset. The algorithm terminates when the log-likelihood $previous_L$ computed in the previous iteration differs from the log-likelihood in the current iteration L, by less than the value specified by the input argument precision. Correctness of the algorithm was verified by comparing its output to the output of MATLAB's function $\mathrm{gmdistribution.fit}$. Here we show only a small part of the

results. To see the full comparison results, the reader is referred to explore the example code in “Chapter 7/7.1 Fitting Mixture of Gaussians/ fit_mog_ex1.m”. Also, the resulting plots are shown in (Figure 7.1) and (Figure 7.2).

```
>> fit_mog_ex1
41 iterations, log-likelihood = -34353.0899      % Result from gmdistribution.fit
30 iterations, log-likelihood = -34353.1         % Result from fit_mog
  0.4943   0.5057                                % λ from gmdistribution.fit
  0.4937   0.5063                                % λ from fit_mog

  1.0145   5.0120                                % means μ from gmdistribution.fit
  1.0053   2.0055

  1.0146   5.0142                                % means μ from fit_mog
  1.0053   2.0069
```

7.2 Fitting the t-distribution

Before talking about fitting the t-distribution, it is useful to familiarize yourself with the univariate t-distribution and the gamma distribution by changing the parameters and observing the corresponding changes. The code for the univariate t-distribution and the gamma distribution is shown below. The plots in (Figure 7.3 and Figure 7.4) were produced by the example code in the files “Chapter 7\7.2 Fitting the t-distribution\pdf_t_ex1.m” and “Chapter 7\7.2 Fitting the t-distribution\pdf_gamma_ex1.m”.

```
% Univariate t-distribution pdf.
function [px] = pdf_t (x, mu, sig, nu)
    px = gamma((nu+1)/2) / (sqrt(nu*pi*sig) * gamma(nu/2));
    px = px * (1 + (x-mu).^2 ./ (nu*sig)) .^ ((-nu-1)/2);
end
```

```
% Gamma distribution pdf.
function [px] = pdf_gamma (x, alpha, beta)
    px = beta^alpha / gamma(alpha);
    px = px * exp(x .* (-beta)) .* x.^(alpha-1);
end
```

Next, we will talk about the pdf of the multivariate t-distribution, shown in mathematical form in “7.5 The t-distribution” [1]. One challenge in writing that formula as MATLAB code is that for big input values the $gamma$ function results in Inf. You can convince yourself by executing this in MATLAB: $gamma(500)$. That is why we must write $exp(gammaln((nu+D)/2) - gammaln(nu/2))$ instead of literally copying the mathematical formula $gamma((nu+D)/2) \,/\, gamma(nu/2)$. According to the MATLAB documentation for the function $gammaln$: “The $gammaln$ command avoids the underflow and overflow that may occur if it is computed directly using $\log(gamma(A))$”. The code for the multivariate t-distribution is shown below.

```
% Multivariate t-distribution pdf.
function [px] = pdf_tm (x, mu, sig, nu)
    D = length(mu);
    % gammaln is used instead of gamma to avoid overflow.
    % gamma((nu+D)/2)/gamma(nu/2) == exp(gammaln((nu+D)/2)-gammaln(nu/2)).
    c = exp(gammaln((nu+D)/2) - gammaln(nu/2));
    c = c / ((nu*pi)^(D/2) * sqrt(det(sig)));

    I = size(x,1);
    delta = zeros (I,1);
    x_minus_mu = bsxfun (@minus, x, mu);
    temp = x_minus_mu * inv(sig);
    for i = 1 : I
        delta(i) = temp(i,:) * x_minus_mu(i,:)';
    end

    px = 1 + (delta ./ nu);
    px = px .^ (-nu-D)/2;
    px = px * c;
end
```

Finally, we present the code for fitting the t-distribution. For the most part, the code is a simple translation of the mathematical formulas to MATLAB code. The code for evaluating the cost function is in the file “Chapter 7\7.2 Fitting the t-distribution\fit_t_cost.m”, and the code for minimizing that cost function is in “Chapter 7\7.2 Fitting the t-distribution\fit_t_minimize_cost.m”. The minimization method that the code in “fit_t_minimize_cost.m” implements is called “Direct Line Search”, and you can read more about it in “Appendix B.3” [1]. The main code is in the file “Chapter 7\7.2 Fitting the t-distribution\fit_t.m”, and it is shown below. The parameter nu is updated in one line by calling the function `fit_t_minimize_cost`, in the main code “fit_t.m”. In (Figure 7.5) you can see a comparison of the normal distribution and the t-distribution, that is how outliers affect each of the two distributions.

```
% Description: Fitting the t-distribution.
% Input: x - a matrix where each row is one datapoint,
%        precision - the algorithm stops when the difference between
%                    the previous and the new likelihood is < precision.
%                    Typically this is a small number like 0.01.
% Output: mu  - the mean of the fitted t-distribution,
%         sig - the scale matrix,
%         nu - degrees of freedom.
function [mu, sig, nu] = fit_t (x, precision)
    % Initialize mu to the mean of the dataset.
    I = size (x, 1);
    dataset_mean = sum(x,1) ./ I;
    mu = dataset_mean;

    % Initialize sig to the covariance of the dataset.
    D = size (x, 2);
    dataset_variance = zeros (D, D);
    x_minus_dataset_mean = bsxfun (@minus, x, dataset_mean);
    for i = 1 : I
        mat = x_minus_dataset_mean(i,:);
        mat = mat' * mat;
        dataset_variance = dataset_variance + mat;
    end
```

```
    dataset_variance = dataset_variance ./ I;
    sig = dataset_variance;

    % Initialize degrees of freedom to 1000 (just a random large value).
    nu = 1000;

    % The main loop.
    iterations = 0;
    previous_L = 1000000; % just a random initialization
    delta = zeros (I,1);
    while true
        % Expectation step.
        % Compute delta.
        x_minus_mu = bsxfun (@minus, x, mu);
        temp = x_minus_mu * inv(sig);
        for i = 1 : I
            delta(i) = temp(i,:) * x_minus_mu(i,:)';
        end

        % Compute E_hi.
        nu_plus_delta = nu + delta;
        E_hi = (nu + D) ./ nu_plus_delta;

        % Compute E_log_hi.
        E_log_hi = psi((nu+D)/2) - log(nu_plus_delta./2);

        % Maximization step.
        % Update mu.
        E_hi_sum = sum (E_hi);
        E_hi_times_xi = bsxfun (@times, E_hi, x);
        mu = sum (E_hi_times_xi, 1) ./ E_hi_sum;

        % Update sig.
        x_minus_mu = bsxfun (@minus, x, mu);
        sig = zeros (D,D);
        for i = 1 : I
            xmm = x_minus_mu(i,:);
            sig = sig + E_hi(i) * xmm' * xmm;
        end
        sig = sig ./ E_hi_sum;

        % Update nu by minimizing a cost function with line search.
        nu = fit_t_minimize_cost (E_hi, E_log_hi);

        % Compute delta again, because the parameters were updated.
        temp = x_minus_mu * inv(sig);
        for i = 1 : I
            delta(i) = temp(i,:) * x_minus_mu(i,:)';
        end

        % Compute the log likelihood L.
        L = I * (gammaln((nu+D)/2) - (D/2)*log(nu*pi) - ...
            log(det(sig))/2 - gammaln(nu/2));
        s = sum (log(1 + delta./nu)) / 2;
```

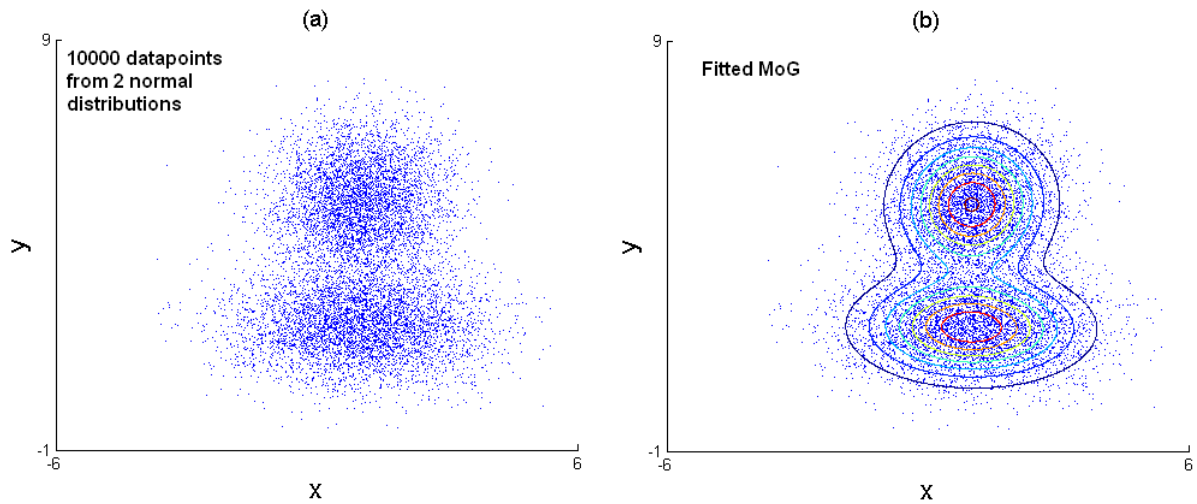


Figure 7.1 The data in (a) was generated from two Gaussian distributions, and then Mixture of Gaussians (MoG) was fitted to the data (b).

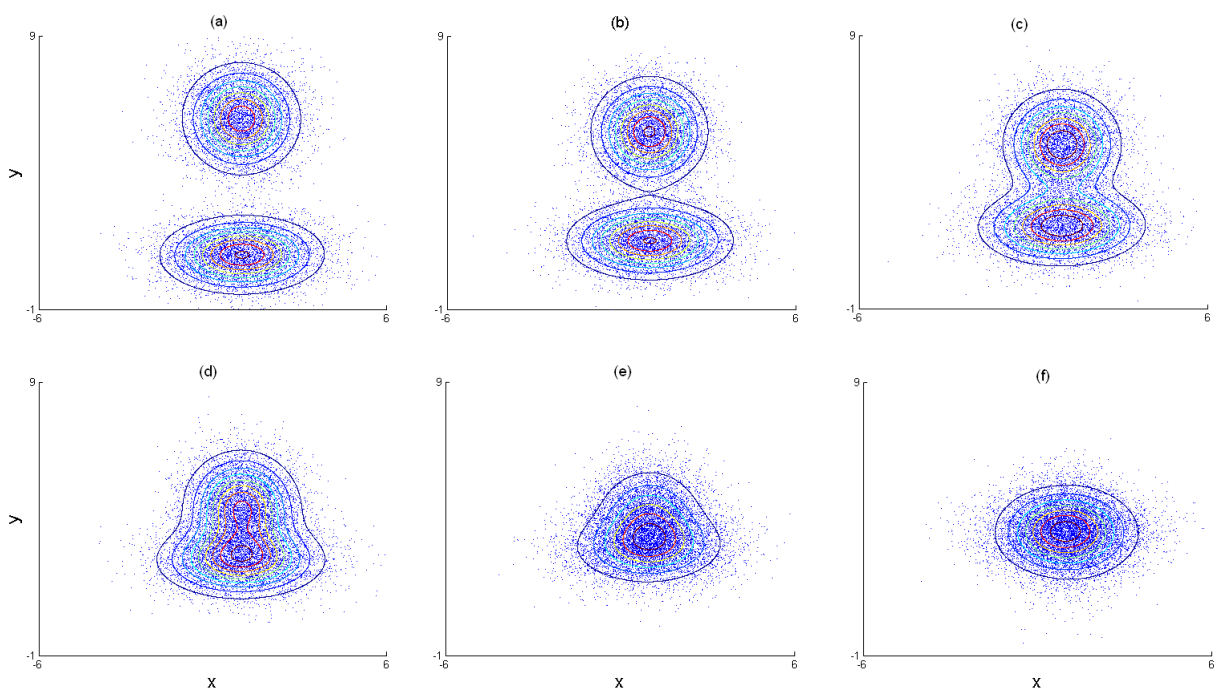


Figure 7.2 For each plot the data was generated from two different normal distributions, by varying the mean. As the means get closer to each other (a-f), the Mixture of Gaussians fit produces a very interesting blending effect. In the beginning (a) the two Gaussians are completely separate and in (b) they make the first contact. They blend into each other (c-e), until they finally become one (f).

```
        L = L - (nu+D)*s;
        iterations = iterations + 1;
        if abs(L - previous_L) < precision
            break;
        end
        previous_L = L;
    end
end
```

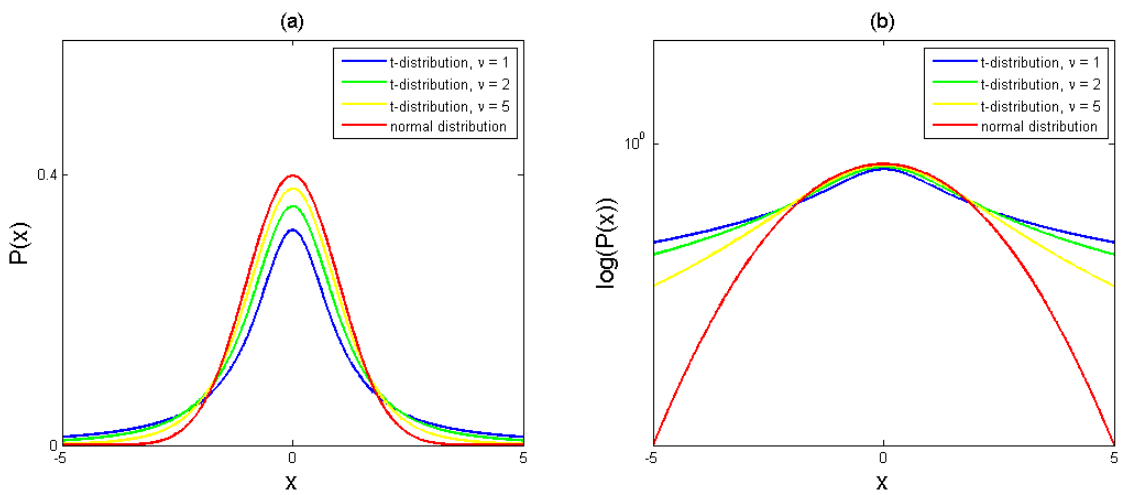


Figure 7.3 (a) As the degrees of freedom parameter ν is increasing, the t-distribution is getting closer to the normal distribution. (b) The differences can be seen better on a logarithmic scale. The corresponding code for this figure can be found in "Chapter 7\7.2 Fitting the t-distribution\pdf_t_ex1.m".

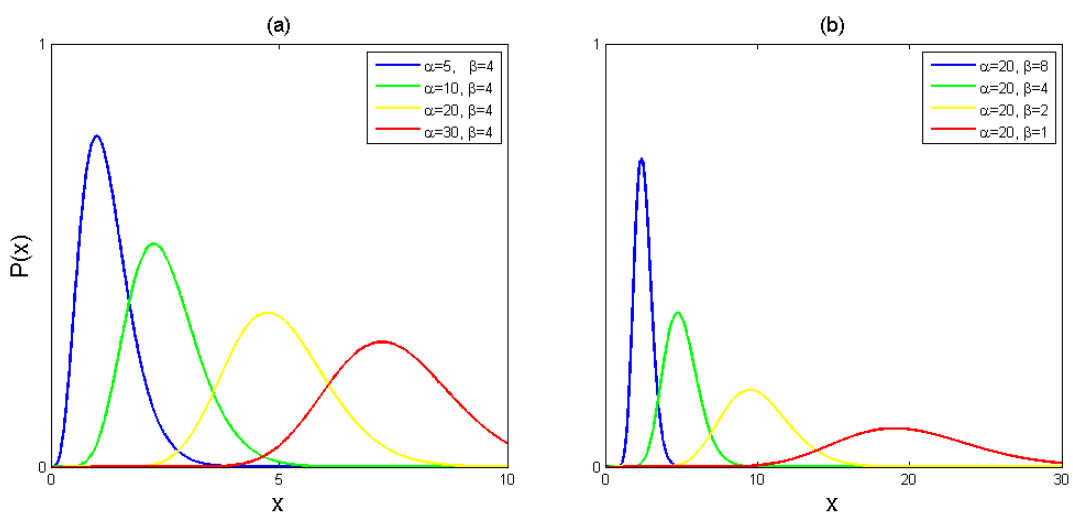


Figure 7.4 Gamma distributions with (a) varying α while keeping β constant and (b) varying β while keeping α constant. The results agree with the formulas for the mean $\frac{\alpha}{\beta}$ and the variance $\frac{\alpha}{\beta^2}$, that is, as α increases (a) both the mean and the variance increase, and as β decreases (b) also both the mean and the variance increase.

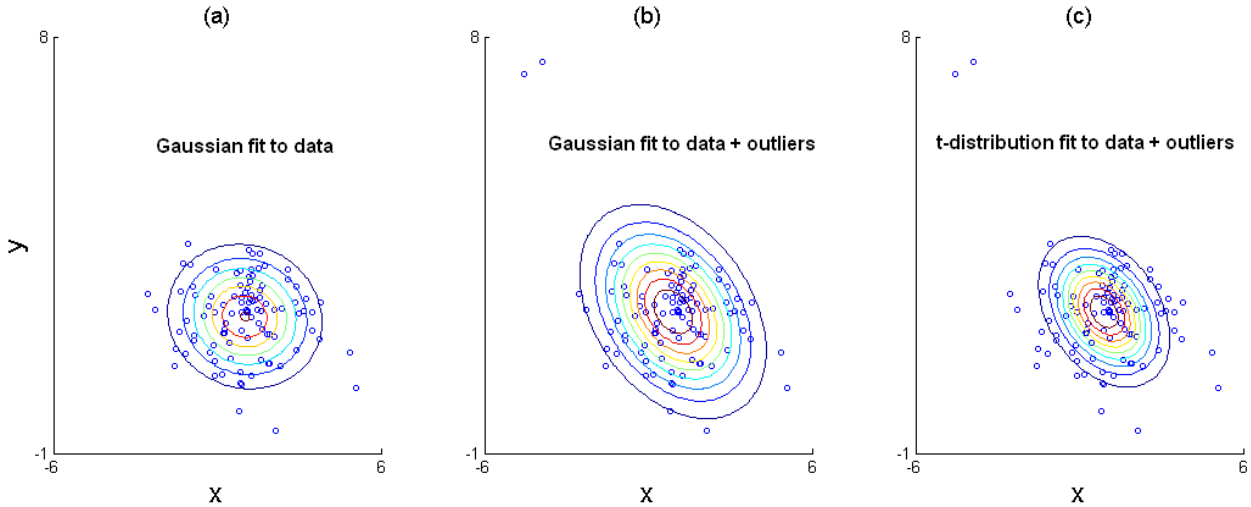


Figure 7.5 (a) One Gaussian is fitted to the data. (b) Few outliers affect the Gaussian fit drastically. (c) The t-distribution is more stable, more resistant to outliers.

7.3 Fitting a factor analyzer

```
% Description: Fitting a factor analyzer.
% Input: x - a matrix where each row is one datapoint,
%        K - number of factors.
% Output: mu - 1xD mean vector,
%         phi - DxK matrix containing K factors in its columns,
%         sig - Dx1 vector representing the DxD diagonal matrix Sigma.
function [mu, phi, sig] = fit_fa (X, K, iterations)
    I = size(X,1);
    D = size(X,2);

    % Initialize mu to the data mean.
    mu = sum(X,1) ./ I;

    %Initialize phi to random values.
    rng('default');
    phi = randn(D,K);

    % Initialize sig, by setting its diagonal elements to the
    % variances of the D data dimensions.
    x_minus_mu = bsxfun (@minus, X, mu);
    sig = sum (x_minus_mu .^ 2, 1) ./ I;

    % The main loop.
    iterations_count = 0;
    while true
        % Expectation step.
        inv_sig = diag(1 ./ sig);
        phi_transpose_times_sig_inv = phi' * inv_sig;
        temp = inv (phi_transpose_times_sig_inv * phi + eye(K));
```

```
        E_hi = temp * phi_transpose_times_sig_inv * x_minus_mu';
        E_hi_hitr = cell (1, I);
        for i = 1 : I
            e = E_hi(:,i);
            E_hi_hitr{i} = temp + e*e';
        end

        % Maximization step.
        % Update phi.
        phi_1 = zeros(D,K);
        for i = 1 : I
            phi_1 = phi_1 + (x_minus_mu(i,:)' * E_hi(:,i)');
        end
        phi_2 = zeros(K,K);
        for i = 1 : I
            phi_2 = phi_2 + E_hi_hitr{i};
        end
        phi_2 = inv (phi_2);
        phi = phi_1 * phi_2;

        % Update sig.
        sig_diag = zeros(D,1);
        for i = 1 : I
            xm = x_minus_mu(i,:)';
            sig_1 = xm .* xm;
            sig_2 = (phi * E_hi(:,i)) .* xm;
            sig_diag = sig_diag + sig_1 - sig_2;
        end
        sig = sig_diag ./ I;

        iterations_count = iterations_count + 1;
        if iterations_count == iterations
            break;
        end
    end
end
```

The example code can be found in “Chapter 7\7.3 Fitting a Factor Analyzer\fit_fa_ex1.m”, and the dataset is in “Datasets/faceNorm”. The results are presented in (Figure 7.6).

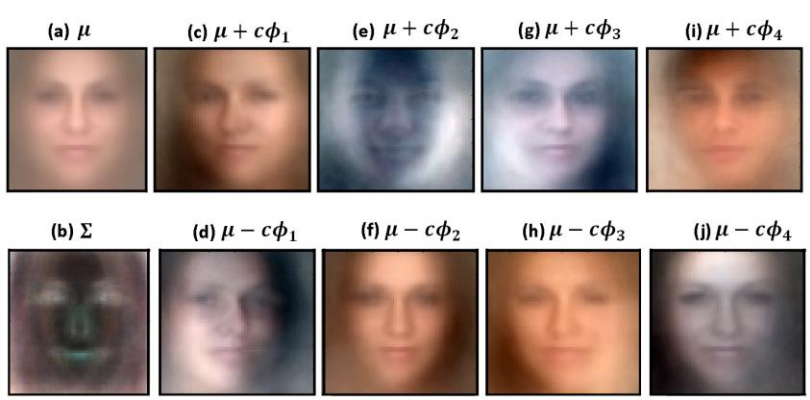


Figure 7.6 (a) The mean face. (b) The covariance component Σ for the face model. (c-j) Linear combinations of the mean face and the first four factors. Each of the factors encodes a mode of variation in the dataset.

Chapter 8

Regression Models

8.1 ML fitting of linear regression model

```
% Description: ML Fitting of linear regression model.
% Input: X - a (D+1)xI data matrix, where D is the data dimensionality
%            and I is the number of training examples. The training
%            examples are in X's columns rather than in the rows, and
%            the first row of X equals ones(1,I).
%        w - a Ix1 vector containing the corresponding world states for
%            each training example.
% Output: phi - a (D+1)x1 vector containing the linear function
coefficients,
%         sig - variance.
function [phi, sig] = fit_lr (X, w)
    % Compute phi.
    phi = inv(X*X')*X*w;

    % Compute sig.
    I = size(X,2);
    temp = w - X'*phi;
    sig = (temp' * temp) / I;
end
```

The example code can be found in “Chapter 8\8.1 ML Fitting of Linear Regression Model\fit_lr_ex1.m”, and the resulting plot is shown in (Figure 8.1).

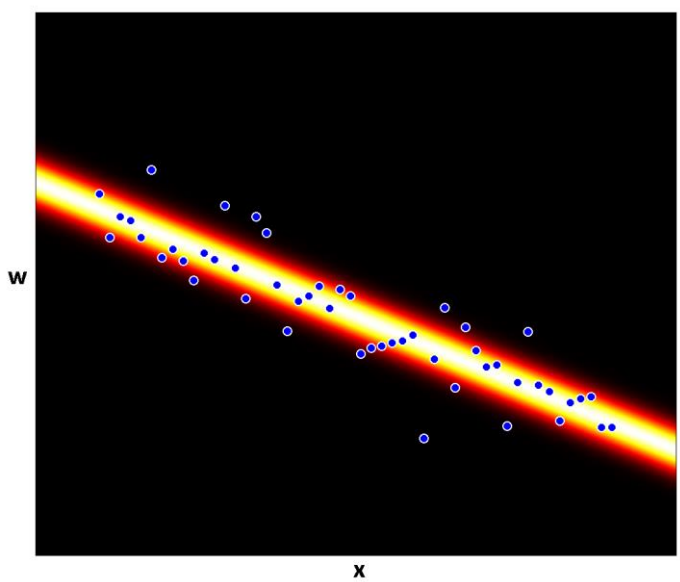


Figure 8.1 A linear regression model fitted to 50 data points.

8.2 Bayesian linear regression

The example code can be found in “Chapter 8\8.2 Bayesian Linear Regression\fit_blr_ex1.m”, and the results are shown in (Figure 8.2) and (Figure 8.3). The variance was computed by using MATLAB’s function $fminbnd$, for which you can read more in MATLAB’s documentation.

```
% Description: Bayesian linear regression.
% Input: X - a (D+1)xI data matrix, where D is the data dimensionality
%            and I is the number of training examples. The training
%            examples are in X's columns rather than in the rows, and
%            the first row of X equals ones(1,I).
%        w - a Ix1 vector containing the corresponding world states for
%            each training example,
%        var_prior - scale factor for the prior spherical covariance,
%        X_test - a data matrix containing training examples for which
%                 we need to make predictions.
% Output: mu_test - a vector of size (I_test x 1), such that mu_test(i)
%                   is the mean of the distribution P(w|x*) for the test
%                   data example x* = X_test(:,i),
%         var_test - a vector of size (I_test x 1), such that var_test(i)
%                    is the variance of the distribution P(w|x*) for the
%                    test data example x* = X_test(:,i),
%         var - can later be used for plotting the posterior over phi,
%         A_inv - can later be used for plotting the posterior over phi.
function [mu_test, var_test, var, A_inv] = fit_blr(X, w, var_prior, X_test)
    D = size(X,1) - 1;
    I = size(X,2);
    I_test = size(X_test,2);

    % Compute the variance. Use the range [0,variance of world values].
    % Constrain var to be positive, by expressing it as var=sqrt(var)^2,
    % that is, the standard deviation squared.
    mu_world = sum(w) / I;
    var_world = sum((w - mu_world) .^ 2) / I;
    var = fminbnd(@(var) fit_blr_cost(var, X, w, var_prior), 0, var_world);

    % Compute A_inv.
    A_inv = 0;
    if D < I
        A_inv = inv ((X*X') ./ var + eye(D+1) ./ var_prior);
    else
        A_inv = eye(D+1) - X*inv(X'*X + (var/var_prior)*eye(I))*X';
        A_inv = var_prior * A_inv;
    end

    % Compute the mean for each test example.
    temp = X_test' * A_inv;
    mu_test = (temp * X * w) ./ var;

    % Compute the variance for each test example.
    var_test = repmat(var,I_test,1);
    for i = 1 : I_test
        var_test(i) = var_test(i) + temp(i,:) * X_test(:,i);
    end
end
```

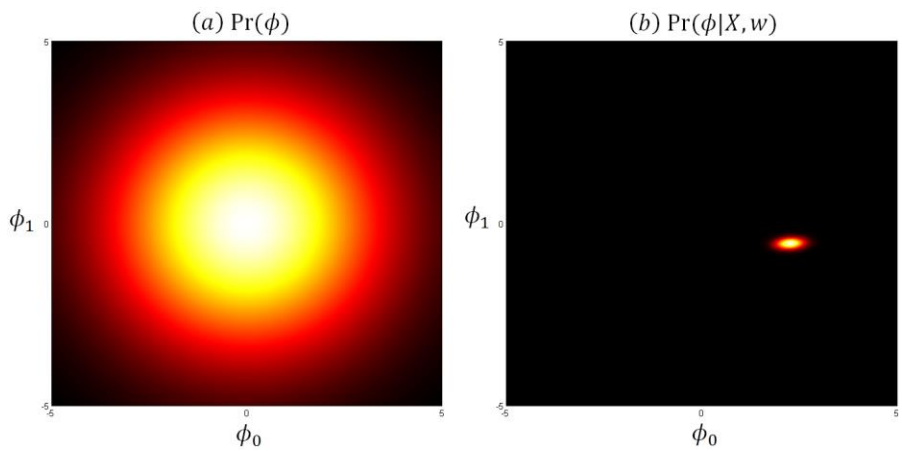


Figure 8.2 (a) The prior $Pr(\phi)$ over the intercept ϕ_0 and the slope ϕ_1, represents our knowledge about these parameters before we observe the data. (b) The posterior $Pr(\phi|X,w)$ which represents our knowledge after observing the data, is much more certain than the prior.

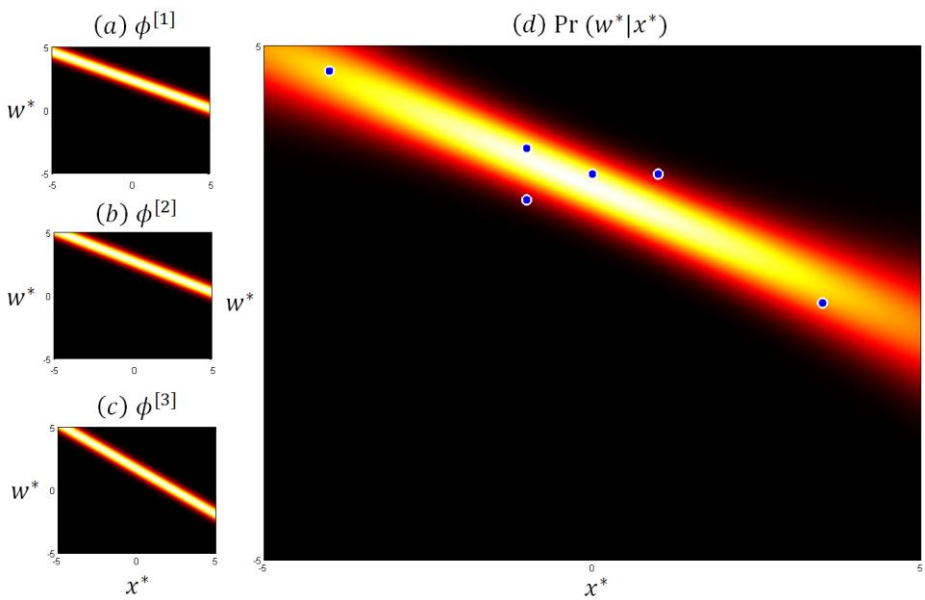


Figure 8.3 (a-c) Three samples from the posterior $P(\phi|X,w)$. (d) The predictive distribution $\mathrm{Pr}(w^*|x^*)$ is formed as an infinite weighted sum of the predictions from all possible parameter settings, where the weight is given by the posterior $P(\phi|X,w)$.

8.3 Gaussian process regression

The example code can be found in "Chapter 8\8.3 Gaussian Process Regression\fit_gpr_ex1.m", and the results are shown in (Figure 8.4). Two kernel functions are available in the same folder: $kernel_gauss.m$ and $kernel_linear.m$. The Gaussian kernel was used in (Figure 8.4). Using the linear kernel results in Bayesian linear regression. Other kernel functions can easily be added if necessary.

```
% Input: X - a (D+1)xI data matrix, where D is the data dimensionality
%            and I is the number of training examples. The training
%            examples are in X's columns rather than in the rows, and
%            the first row of X equals ones(1,I).
%        w - a Ix1 vector containing the corresponding world states for
%            each training example,
%        var_prior - scale factor for the prior spherical covariance,
%        X_test - a data matrix containing training examples for which
%                 we need to make predictions,
%        kernel - the kernel function.
% Output: mu_test - a vector of size (I_test x 1), such that mu_test(i)
%                   is the mean of the distribution P(w|x*) for the test
%                   data example x* = X_test(:,i),
%         var_test - a vector of size (I_test x 1), such that var_test(i)
%                    is the variance of the distribution P(w|x*) for the
%                    test data example x* = X_test(:,i).
function [mu_test, var_test] = fit_gpr (X, w, var_prior, X_test, kernel)
    I = size(X,2);
    I_test = size(X_test,2);

    % Compute K[X,X].
    K = zeros(I,I);
    for i=1:I
        for j=1:I
            K(i,j) = kernel(X(:,i), X(:,j));
        end
    end

    % Compute K[X_test,X].
    K_test = zeros(I_test, I);
    for i=1:I_test
        for j=1:I
            K_test(i,j) = kernel(X_test(:,i), X(:,j));
        end
    end

    % Compute the variance. Use the range [0,variance of world values].
    % Constrain var to be positive, by expressing it as var=sqrt(var)^2,
    % that is, the standard deviation squared.
    mu_world = sum(w) / I;
    var_world = sum((w - mu_world) .^ 2) / I;
    var = fminbnd (@(var) fit_gpr_cost (var, K, w, var_prior),0,var_world);

    % Compute A_inv.
    A_inv = inv (K + (var/var_prior)*eye(I));

    % Compute the mean for each test example.
    mu_temp_1 = K_test * w;
    K_test_A_inv = K_test * A_inv;
    mu_temp_2 = K_test_A_inv * K * w;
    c = var_prior/var;
    mu_test = c * (mu_temp_1 - mu_temp_2);
```

```
    % Compute the variance for each test example.
    var_test = repmat(var,I_test,1);
    for i = 1 : I_test
        x_star = X_test(:,i);
        part1 = kernel(x_star, x_star);
        part2 = K_test_A_inv(i,:) * K_test(i,:)';
        var_test(i) = var_test(i) + var_prior * (part1 - part2);
    end
end
```

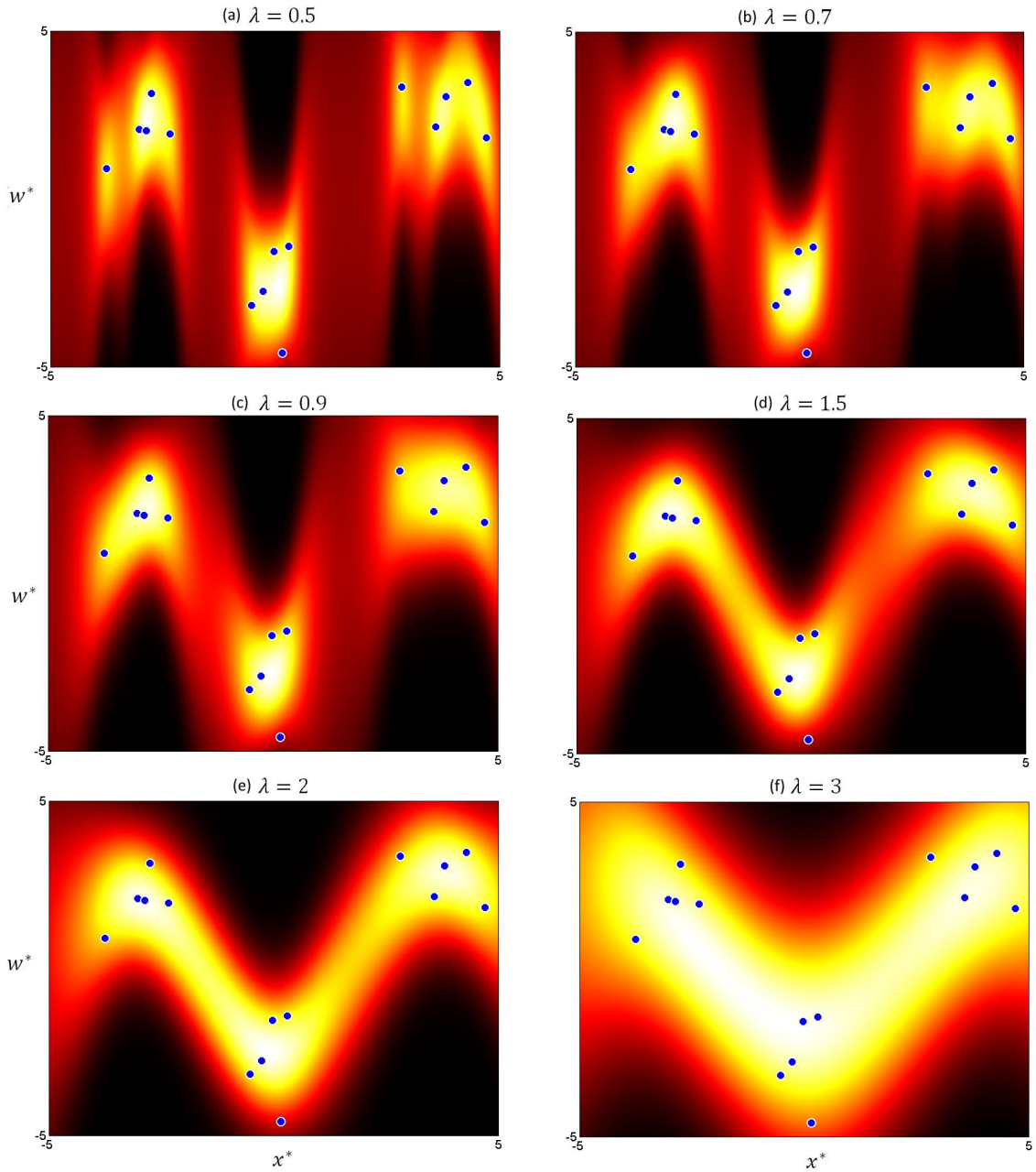


Figure 8.4 Gaussian process regression with a Gaussian kernel. When the parameter λ is small (a-b), the distribution contains isolated island-like regions at the data points. As λ increases (c-f) the isolated regions blend into each other.

8.4 Sparse linear regression

Unlike Bayesian linear regression which uses a normal prior, sparse linear regression uses a product of 1D t-distributions which encourages sparse solutions. The multivariate t-distribution is similar to the multivariate normal distribution, but it has longer tails. It is important to understand that the product of 1D t-distributions is different than the multivariate t-distribution (Figure 8.5).

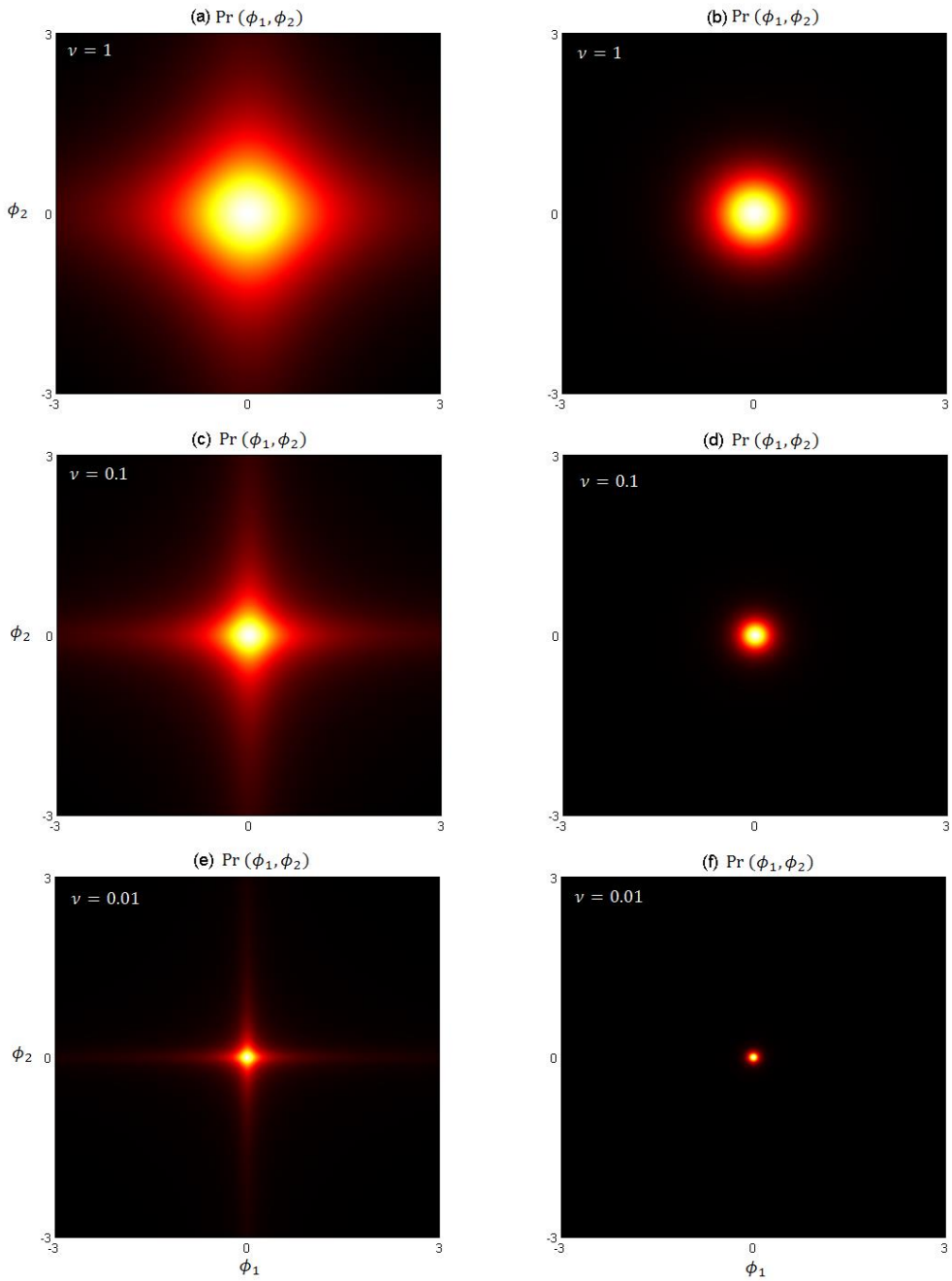


Figure 8.5 A comparison between the product of two 1D t-distributions and the multivariate t-distribution. The parameters μ and σ^2 are fixed: $\mu = 0$, $\sigma^2 = 1$ for the 1D t-distributions, and $\mu = [0\ 0]$, $\sigma^2 = [1\ 0; 0\ 1]$ for the multivariate t-distribution. The parameter ν varies in each row. The left column (a,c,e) contains products of 1D t-distributions, while the right column (b,d,f) contains multivariate t-distributions. We can conclude that the product of 1D t-distributions is different than the corresponding multivariate t-distribution.

The example code for (Figure 8.5) is in “Chapter 8\8.4 Sparse Linear Regression\td_ex1.m”, and the example code for (Figure 8.6) is in “Chapter 8\8.4 Sparse Linear Regression\fit_slr_ex1.m”.

```
% Description: Sparse linear regression.
% Input: X - a (D+1)xI data matrix, where D is the data dimensionality
%            and I is the number of training examples. The training
%            examples are in X's columns rather than in the rows, and
%            the first row of X equals ones(1,I).
%        w - a Ix1 vector containing the corresponding world states for
%            each training example,
%        nu - degrees of freedom typically nu<0.001,
%        X_test - a data matrix containing training examples for which
%                 we need to make predictions.
% Output: mu_test - a vector of size (I_test x 1), such that mu_test(i)
%                   is the mean of the distribution P(w|x*) for the test
%                   data example x* = X_test(:,i),
%         var_test - a vector of size (I_test x 1), such that var_test(i)
%                    is the variance of the distribution P(w|x*) for the
%                    test data example x* = X_test(:,i).
function [mu_test, var_test] = fit_slr (X, w, nu, X_test)
    D = size(X,1) - 1;
    I = size(X,2);
    I_test = size(X_test,2);

    % Initialize H.
    H = ones(D+1,1);
    H_old = zeros(D+1,1);

    % Precompute. No need to compute these each iteration.
    X_w = X*w;
    X_t = X';
    X_Xt = X*X_t;

    % The main loop.
    iterations_count = 0;
    precision = 0.0001;
    while true
        % Compute the variance. Use the range [0,variance of world values].
        % Constrain var to be positive, by expressing it as
        % var=sqrt(var)^2, that is, the standard deviation squared.
        mu_world = sum(w) / I;
        var_world = sum((w - mu_world) .^ 2) / I;
        var = fminbnd (@(var) fit_slr_cost (var, X, w, H), 0, var_world);

        % Update sig and mu.
        sig = inv(X_Xt/var + diag(H));
        mu = (sig * X_w) / var;

        % Update H.
        H = H .* diag(sig);
        H = nu + 1 - H;
        H = H ./ (mu.^2 + nu);
        H(1) = 1; % make sure the first dimension stays constant.
```

```
        iterations_count = iterations_count + 1;
        stop = all(abs(H-H_old) < precision);
        if stop == true
            break;
        end

        % Save H for the next iteration.
        H_old = H;
    end

    % Prune step. Remove row d in X and element d in H, if H(d)>1000.
    selector = H < 1;
    X = X(selector,:);
    X_test = X_test(selector,:);
    H = H(selector);

    % Compute A_inv.
    H_inv = diag(1 ./ H);
    H_inv_X = H_inv * X;
    temp = inv(X' * H_inv_X + var*eye(I));
    A_inv = H_inv - H_inv_X * temp * X' * H_inv;

    % Compute the mean for each test example.
    temp2 = X_test' * A_inv;
    mu_test = (temp2 * X * w) ./ var;

    % Compute the variance for each test example.
    var_test = repmat(var,I_test,1);
    for i = 1 : I_test
        var_test(i) = var_test(i) + temp2(i,:) * X_test(:,i);
    end
end
```

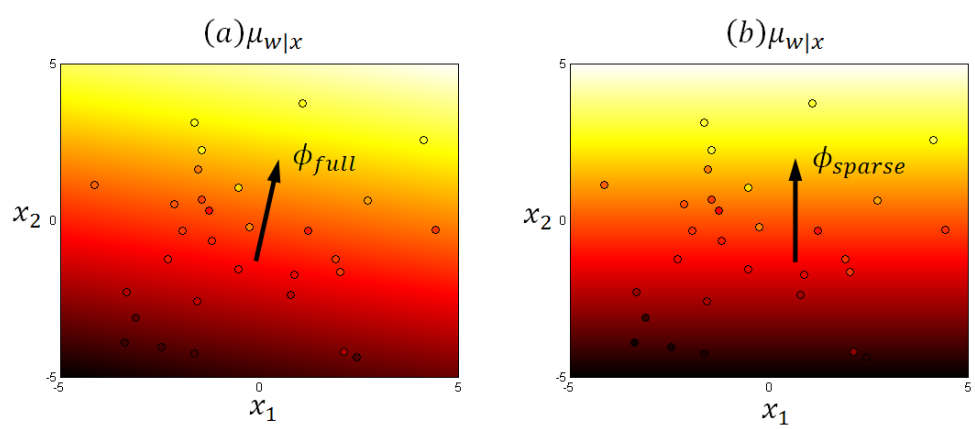


Figure 8.6 The background color represents the mean $\mu_{w|x}$ of the Gaussian prediction $\Pr(w|x)$ for x. The color of the data points represents the value for w. (a) Bayesian linear regression model fit whose predictions depend on both dimensions x_1 and x_2. (b) Sparse linear regression model fit whose predictions depend only on one dimension x_2.

8.5 Dual Bayesian linear regression

The example code is in “Chapter 8\8.5 Dual Bayesian Linear Regression\ fit_dr_ex1.m”.

```
% Description: Dual regression.
% Input: X - a (D+1)xI data matrix, where D is the data dimensionality
%            and I is the number of training examples. The training
%            examples are in X's columns rather than in the rows, and
%            the first row of X equals ones(1,I),
%        w - a Ix1 vector containing the corresponding world states for
%            each training example,
%        var_prior - scale factor for the prior spherical covariance,
%        X_test - a data matrix containing training examples for which
%                 we need to make predictions,
%        kernel - the kernel function.
% Output: mu_test - a vector of size (I_test x 1), such that mu_test(i)
%                   is the mean of the distribution P(w|x*) for the test
%                   data example x* = X_test(:,i),
%         var_test - a vector of size (I_test x 1), such that var_test(i)
%                    is the variance of the distribution P(w|x*) for the
%                    test data example x* = X_test(:,i).
function [mu_test, var_test] = fit_dr (X, w, var_prior, X_test, kernel)
    I = size(X,2);
    I_test = size(X_test,2);

    % Compute K[X,X].
    K = zeros(I,I);
    for i=1:I
        for j=1:I
            K(i,j) = kernel(X(:,i), X(:,j));
        end
    end

    % Compute K[X_test,X].
    K_test = zeros(I_test, I);
    for i=1:I_test
        for j=1:I
            K_test(i,j) = kernel(X_test(:,i), X(:,j));
        end
    end

    % Compute the variance. Use the range [0,variance of world values].
    % Constrain var to be positive, by expressing it as var=sqrt(var)^2,
    % that is, the standard deviation squared.
    mu_world = sum(w) / I;
    var_world = sum((w - mu_world) .^ 2) / I;
    var = fminbnd (@(var)fit_dr_cost (var, K, w, var_prior), 0, var_world);

    % Compute A_inv.
    A_inv = inv (K*K/var + eye(I)/var_prior);

    % Compute mu_test.
    temp = K_test * A_inv;
    mu_test = (temp*K*w)/var;
```

```
    % Compute var_test.
    var_test = repmat(var,I_test,1);
    for i = 1 : I_test
        var_test(i) = var_test(i) + temp(i,:) * K_test(i,:)';
    end
end
```

8.7 Relevance vector regression

The example code is in “Chapter 8\8.7 Relevance Vector Regression\fit_rvr_ex1.m”, and the result is shown in (Figure 8.7).

```
% Description: Relevance vector regression.
% Input: X - a (D+1)xI data matrix, where D is the data dimensionality
%            and I is the number of training examples. The training
%            examples are in X's columns rather than in the rows, and
%            the first row of X equals ones(1,I).
%        w - a Ix1 vector containing the corresponding world states for
%            each training example,
%        nu - degrees of freedom typically nu<0.001,
%        X_test - a data matrix containing training examples for which
%                 we need to make predictions,
%        kernel - the kernel function.
% Output: mu_test - a vector of size (I_test x 1), such that mu_test(i)
%                   is the mean of the distribution P(w|x*) for the test
%                   data example x* = X_test(:,i),
%         var_test - a vector of size (I_test x 1), such that var_test(i)
%                    is the variance of the distribution P(w|x*) for the
%                    test data example x* = X_test(:,i),
%         relevant_points - Ix1 boolean vector where a 1 at position i
%                           indicates that point X(:,i) remained after
%                           the elimination phase, that is, it is relevant.
function [mu_test, var_test, relevant_points] = ...
         fit_rvr (X, w, nu, X_test, kernel)
    I = size(X,2);
    I_test = size(X_test,2);

    % Compute K[X,X].
    K = zeros(I,I);
    for i=1:I
        for j=1:I
            K(i,j) = kernel(X(:,i), X(:,j));
        end
    end

    % Initialize H.
    H = ones(I,1);
    H_old = zeros(I,1);

    % Pre-compute so that it is not computed each iteration.
    K_K = K*K;
    K_w = K*w;
```

```
% The main loop.
iterations_count = 0;
precision = 0.001;
while true
    % Compute the variance. Use the range [0,variance of world values].
    % Constrain var to be positive, by expressing it as
    % var=sqrt(var)^2, that is, the standard deviation squared.
    mu_world = sum(w) / I;
    var_world = sum((w - mu_world) .^ 2) / I;
    var = fminbnd (@(var) fit_rvr_cost (var, K, w, H), 0, var_world);

    % Update sig and mu.
    sig = inv (K_K/var + diag(H));
    mu = sig*K_w/var;

    % Update H.
    H = H .* diag(sig);
    H = nu + 1 - H;
    H = H ./ (mu.^2 + nu);

    stop = all(abs(H-H_old) < precision);
    if stop == true
        break;
    end

    % Save H for the next iteration.
    H_old = H;
end

% Prune step. Remove column t in X, row t in w, and element t in H,
% if H(t) > 1.
selector = H < 1;
X = X(:,selector);
w = w(selector);
H = H(selector);
relevant_points = selector;

% Recompute K[X,X].
I = size(X,2);
K = zeros(I,I);
for i=1:I
    for j=1:I
        K(i,j) = kernel(X(:,i), X(:,j));
    end
end

% Compute K[X_test,X].
K_test = zeros(I_test, I);
for i=1:I_test
    for j=1:I
        K_test(i,j) = kernel(X_test(:,i), X(:,j));
    end
end

% Compute A_inv.
A_inv = inv (K*K/var + diag(H));
```

```
    % Compute the mean for each test example.
    temp = K_test*A_inv;
    mu_test = temp*K*w/var;

    % Compute the variance for each test example.
    var_test = repmat(var,I_test,1);
    for i = 1 : I_test
        var_test(i) = var_test(i) + temp(i,:)*K_test(i,:)';
    end
end
```

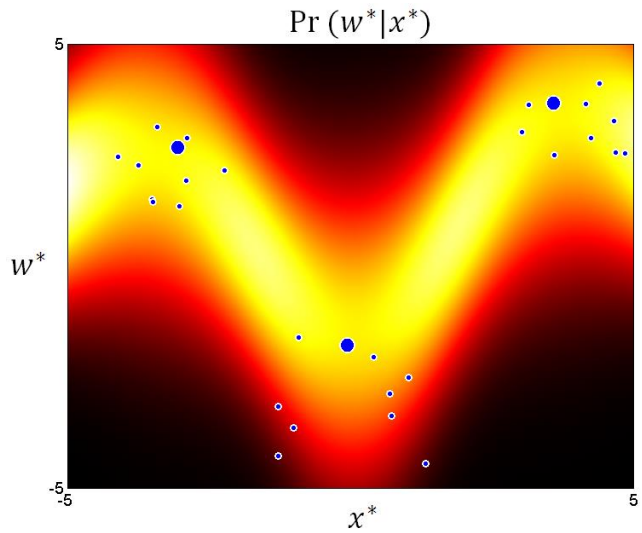


Figure 8.7 Relevance vector regression. The predictive distribution depends only on a subset of the data points, in this case the three larger points. Notice that only one point was picked to represent the corresponding cluster of points.

Chapter 9

Classification Models

9.1 Logistic regression

The main code for logistic regression is in “Chapter 9\9.1 MAP Logistic Regression\fit_logr.m”, and the cost function is in “Chapter 9\9.1 MAP Logistic Regression\fit_logr_cost.m”. Both are shown below.

```
% Description: MAP logistic regression.
% Input: X - a (D+1)xI data matrix, where D is the data dimensionality
%            and I is the number of training examples. The training
%            examples are in X's columns rather than in the rows, and
%            the first row of X equals ones(1,I).
%        w - a Ix1 vector containing the corresponding world states for
%            each training example,
%        var_prior - scale factor for the prior spherical covariance,
%        X_test - a data matrix containing training examples for which
%                 we need to make predictions,
%        initial_phi - (D+1)x1 vector that represents the start solution.
% Output: predictions - 1xI_test row vector which contains the predicted
%                       class values for the input data in X_test.
%         phi - (D+1)x1 column vector that contains the coefficients for
%               the linear activation function.
function [predictions, phi] = fit_logr (X, w, var_prior, X_test,
                                        initial_phi)
    % Find the MAP estimate of the parameters phi.
    options = optimset('GradObj','on','Hessian','on');
    phi = fminunc(@(phi) fit_logr_cost(phi, X, w, var_prior), ...
                  initial_phi, options);

    % Compute the predictions for X_test.
    predictions = sigmoid(phi' * X_test);
end
```

```
% Description: Cost function for MAP logistic regression.
% Input: phi - (D+1)x1 column vector that contains the coefficients for
%              the linear activation function,
%        X - a (D+1)xI data matrix, where D is the data dimensionality
%            and I is the number of training examples. The training
%            examples are in X's columns rather than in the rows, and
%            the first row of X equals ones(1,I).
%        w - a Ix1 vector containing the corresponding world states for
%            each training example,
%        var_prior - scale factor for the prior spherical covariance.
% Output: L - the value of the cost function evaluated at phi,
%         g - (D+1)x1 gradient vector,
%         H - (D+1)x(D+1) Hessian matrix containing the second derivatives.
function [L, g, H] = fit_logr_cost (phi, X, w, var_prior)
    I = size(X,2);
    D = size(X,1) - 1;

    % Initialize.
    L = I * (-log (mvnpdf (phi, zeros(D+1,1), var_prior*eye(D+1))));
    g = I * phi / var_prior;
    H = I * diag(repmat(1/var_prior,1,D+1));
```

```
    predictions = sigmoid(phi' * X);
    for i = 1 : I
        % Update L.
        y = predictions(i);
        if w(i) == 1
            L = L - log(y);
        else
            L = L - log(1-y);
        end

        % Update g and H.
        x_i = X(:,i);
        g = g + (y-w(i)) * x_i;
        H = H + y * (1-y) * (x_i * x_i');
    end
end
```

The figures below are self-explanatory. For each figure we point the reader to the code that generated it.

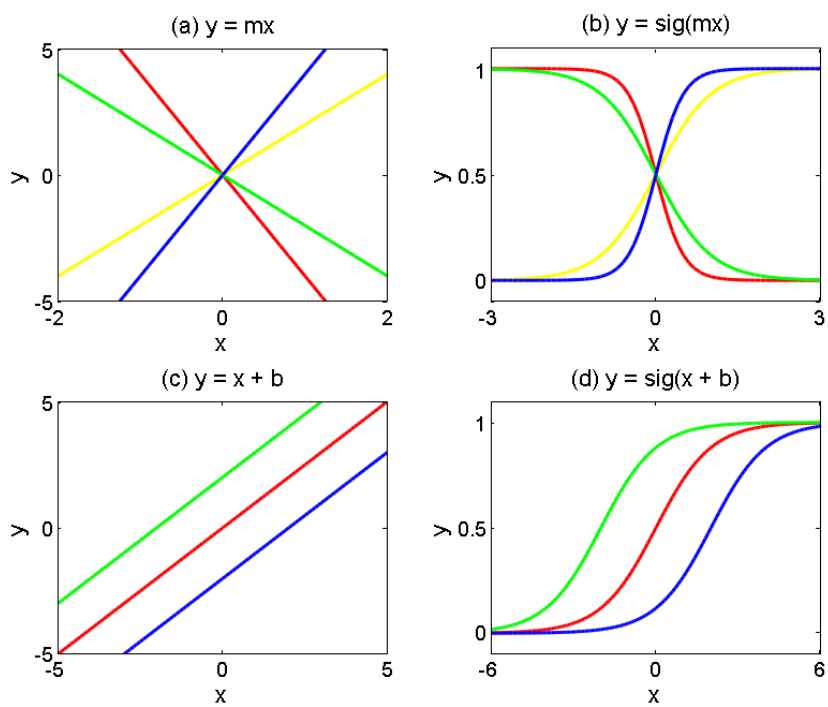


Figure 9.1 Relation between a line and the sigmoid of the line. (a) Four lines with different slopes, and (b) the corresponding sigmoids. Notice that the bigger the slope a line has, the steeper the corresponding sigmoid will be. If the slope of the line is negative, the sigmoid goes from 1 to 0 (from left to right on the x axis), and if the slope of the line is positive, the sigmoid goes from 0 to 1 (from left to right on the x axis). (c) This time we modify the y-intercept and the slope is fixed, and (d) the corresponding sigmoids are moved accordingly along the x-axis. If the y-intercept of the line was moved to the left, the corresponding sigmoid will also move to the left by the same amount. The code that created this figure can be found in “Chapter 9\9.1 MAP Logistic Regression\fit_logr_ex1.m”.

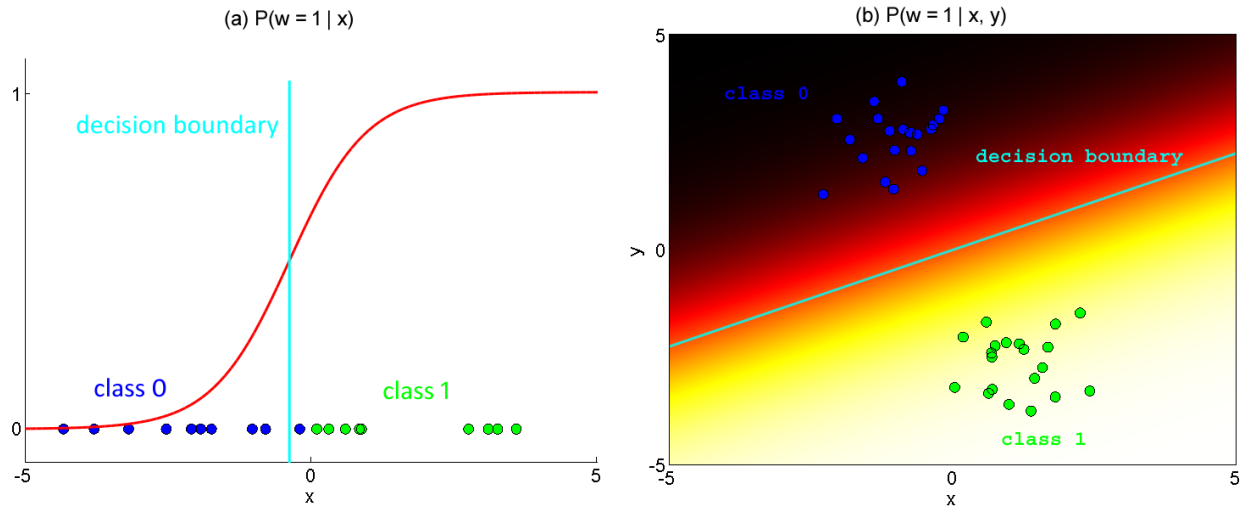


Figure 9.2 Logistic regression model fitted to (a) 1-dimensional and (b) 2-dimensional datasets. Data points in class 0 are colored blue, while data points in class 1 are colored green. The color of the decision boundary is cyan.

9.2 Bayesian logistic regression

The code for the Bayesian logistic regression algorithm is presented here, followed by (Figure 9.3) where the algorithm was used.

```
% Description: Bayesian logistic regression.
% Input: X - a (D+1)xI data matrix, where D is the data dimensionality
%            and I is the number of training examples. The training
%            examples are in X's columns rather than in the rows, and
%            the first row of X equals ones(1,I).
%        w - a Ix1 vector containing the corresponding world states for
%            each training example,
%        var_prior - scale factor for the prior spherical covariance,
%        X_test - a data matrix containing training examples for which
%                 we need to make predictions,
%        initial_phi - (D+1)x1 vector that represents the start solution.
% Output: predictions - 1xI_test row vector which contains the predicted
%                       class values for the input data in X_test.
%         phi - (D+1)x1 column vector that contains the coefficients for
%               the linear activation function.
function [predictions, phi] = fit_blogr (X, w, var_prior, X_test,
initial_phi)
    I = size(X,2);
    I_test = size(X_test,2);
    D = size(X,1) - 1;

    % Find the MAP estimate of the parameters phi.
    options = optimset('GradObj','on','Hessian','on');
    phi = fminunc(@(phi) fit_logr_cost(phi, X, w, var_prior), ...
                  initial_phi, options);
```

```
    % Compute the Hessian at phi.
    H = diag(repmat(1/var_prior,1,D+1));
    ys = sigmoid(phi' * X);
    for i=1:I
        x_i = X(:,i);
        y = ys(i);
        H = H - y * (1-y) * (x_i * x_i');
    end

    % Set the mean and variance of the Laplace approximation.
    mu = phi;
    var = -inv(H);

    % Compute mean and variance of the activation.
    mu_a = mu' * X_test;
    var_a_temp = X_test' * var;
    var_a = zeros(1,I_test);
    for i=1:I_test
        var_a(i) = var_a_temp(i,:) * X_test(:,i);
    end

    % Approximate the integral to get the Bernoulli parameter.
    lambda = sqrt(1 + pi * var_a / 8);
    lambda = mu_a ./ lambda;
    lambda = sigmoid(lambda);

    predictions = lambda;
end
```

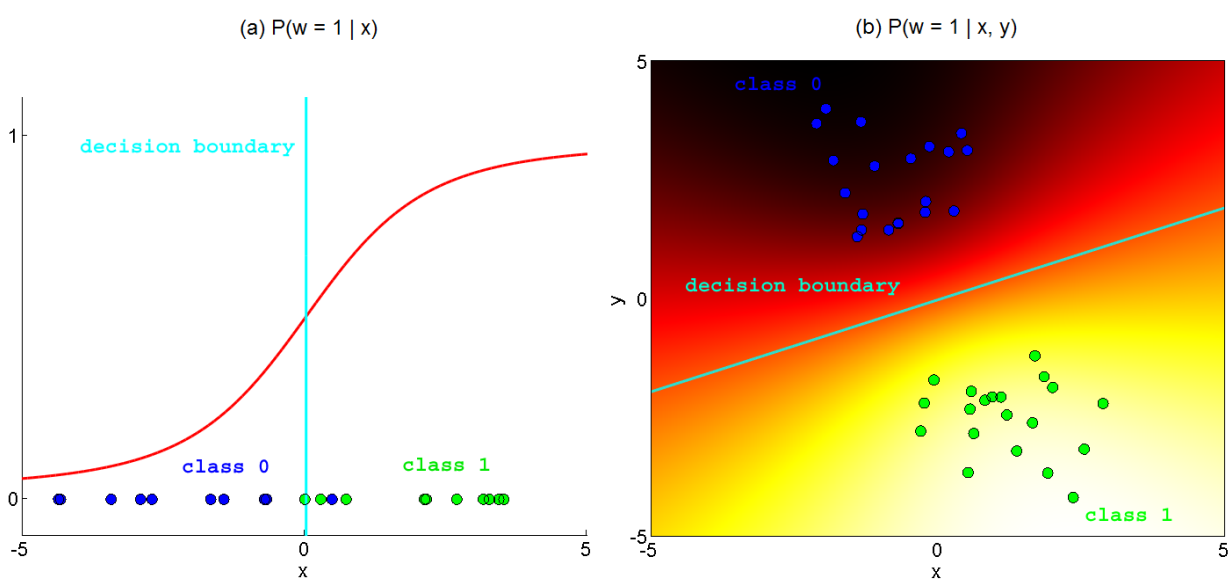


Figure 9.3 Bayesian logistic regression model fitted to (a) 1-dimensional and (b) 2-dimensional datasets. Data points in class 0 are colored blue, while data points in class 1 are colored green. The color of the decision boundary is cyan.

9.3 MAP dual logistic regression

The code for the dual logistic regression algorithm is presented here, followed by (Figure 9.4) which is very simiar to the results obtained from linear logistic regression (Figure 9.2). The function fit_dlogr is very simple. The main work is done in the cost function fit_dlogr_cost.

```
% Description: MAP dual logistic regression.
% Input: X - a (D+1)xI data matrix, where D is the data dimensionality
%            and I is the number of training examples. The training
%            examples are in X's columns rather than in the rows, and
%            the first row of X equals ones(1,I).
%        w - a Ix1 vector containing the corresponding world states for
%            each training example,
%        var_prior - scale factor for the prior spherical covariance,
%        X_test - a data matrix containing training examples for which
%                 we need to make predictions,
%        initial_psi - Ix1 vector that represents the start solution.
% Output: predictions - 1xI_test row vector which contains the predicted
%                       class values for the input data in X_test.
%         psi - Ix1 column vector that contains the coefficients for
%               the activation function.
function [predictions, psi] = fit_dlogr (X,w,var_prior,X_test,initial_psi)
    % Find the MAP estimate of the parameters psi.
    options = optimset('GradObj','on','Hessian','on');
    psi = fminunc(@(psi) fit_dlogr_cost(psi, X, w, var_prior), ...
                  initial_psi, options);

    % Compute the predictions for X_test.
    predictions = sigmoid((X*psi)' * X_test);
end
```

```
% Description: Cost function for MAP dual logistic regression.
% Input: psi - Ix1 column vector that contains the coefficients for
%              the activation function,
%        X - a (D+1)xI data matrix, where D is the data dimensionality
%            and I is the number of training examples. The training
%            examples are in X's columns rather than in the rows, and
%            the first row of X equals ones(1,I).
%        w - a Ix1 vector containing the corresponding world states for
%            each training example,
%        var_prior - scale factor for the prior spherical covariance.
% Output: L - the value of the cost function evaluated at psi,
%         g - Ix1 gradient vector,
%         H - IxI Hessian matrix containing the second derivatives.
function [L, g, H] = fit_dlogr_cost (psi, X, w, var_prior)
    I = size(X,2);
    D = size(X,1) - 1;

    % Initialize.
    L = I * (-log (mvnpdf (psi, zeros(I,1), var_prior*eye(I))));
    g = I * psi / var_prior;
    H = I * diag(repmat(1/var_prior,1,I));
```

```
    predictions = sigmoid((X*psi)' * X);
    for i = 1 : I
        % Update L.
        y = predictions(i);
        if w(i) == 1
            L = L - log(y);
        else
            L = L - log(1-y);
        end

        % Update g and H.
        temp = X' * X(:,i);
        g = g + (y-w(i)) * temp;
        H = H + y * (1-y) * (temp * temp');
    end
end
```

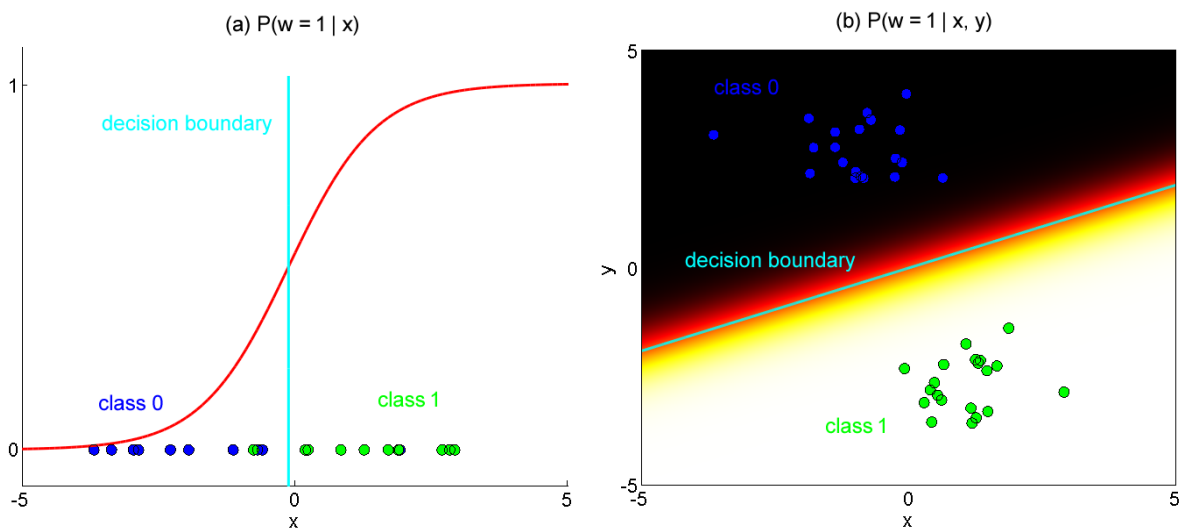


Figure 9.4 Dual logistic regression model fitted to (a) 1-dimensional and (b) 2-dimensional datasets. Data points in class 0 are colored blue, while data points in class 1 are colored green. The color of the decision boundary is cyan.

9.4a Dual Bayesian logistic regression

This algorithm uses the same code as (9.3 MAP Dual Logistic Regression) for computing the parameters ψ. The difference between the two algorithms is in the code after the computation of the parameters ψ. So, (9.4a Dual Bayesian Logistic Regression) can be thought of as an extension of (9.3 MAP Dual Logistic Regression).

```
% Description: Dual Bayesian logistic regression.
% Input: X - a (D+1)xI data matrix, where D is the data dimensionality
%            and I is the number of training examples. The training
%            examples are in X's columns rather than in the rows, and
%            the first row of X equals ones(1,I),
%        w - a Ix1 vector containing the corresponding world states for
%            each training example,
%        var_prior - scale factor for the prior spherical covariance,
%        X_test - a data matrix containing training examples for which
%                 we need to make predictions,
%        initial_psi - Ix1 vector that represents the start solution.
% Output: predictions - 1xI_test row vector which contains the predicted
%                       class values for the input data in X_test.
%         psi - Ix1 column vector that contains the coefficients for
%               the activation function.
function [predictions, psi] = fit_dblogr (X,w,var_prior,X_test,initial_psi)
    % Find the MAP estimate of the parameters psi.
    options = optimset('GradObj','on','Hessian','on');
    psi = fminunc(@(psi) fit_dlogr_cost(psi, X, w, var_prior), ...
                  initial_psi, options);

    % Compute the Hessian at psi.
    I = size(X,2);
    H = diag(repmat(1/var_prior,1,I));
    ys = sigmoid((X*psi)' * X);
    for i=1:I
        x_i = X(:,i);
        y = ys(i);
        H = H - y * (1-y) * X' * x_i * x_i' * X;
    end

    % Set the mean and variance of the Laplace approximation.
    mu = psi;
    var = -inv(H);

    % Compute mean and variance of the activation.
    mu_a_temp = X' * X_test;
    mu_a = mu' * mu_a_temp;
    var_a_temp = X * var * mu_a_temp;
    I_test = size(X_test,2);
    var_a = zeros(1,I_test);
    for i=1:I_test
        var_a(i) = X_test(:,i)' * var_a_temp(:,i);
    end

    % Approximate the integral to get the Bernoulli parameter.
    lambda = sqrt(1 + pi .* var_a ./ 8);
    lambda = mu_a ./ lambda;
    lambda = sigmoid(lambda);

    predictions = lambda;
end
```

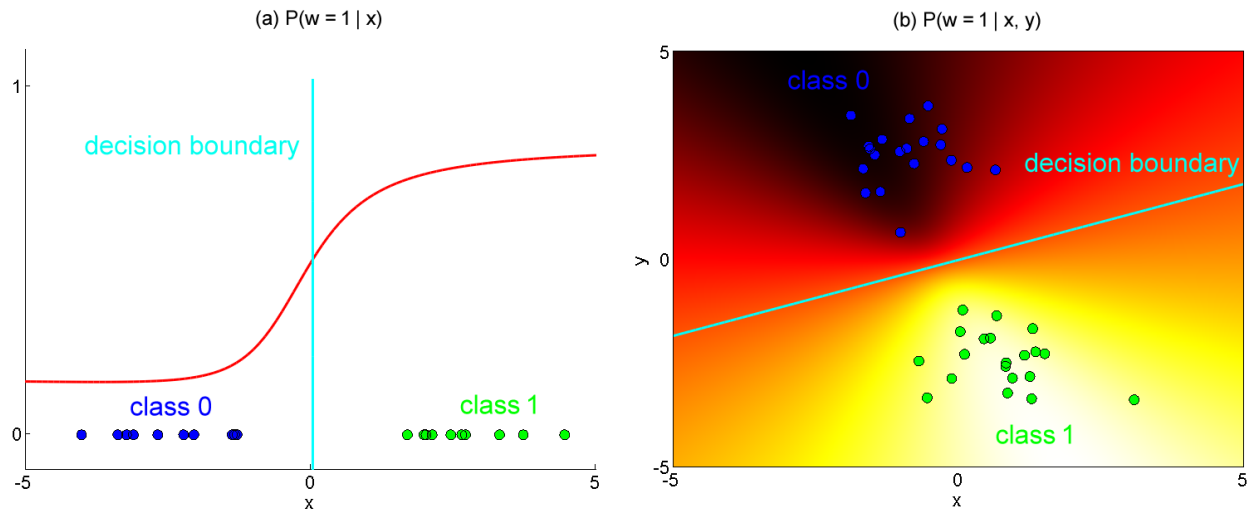


Figure 9.5 Dual Bayesian logistic regression model fitted to (a) 1-dimensional and (b) 2-dimensional datasets. Data points in class 0 are colored blue, while data points in class 1 are colored green. The color of the decision boundary is cyan.

9.4b Gaussian process classification (or kernel logistic regression)

```
% Description: Kernel logistic regression.
% Input: X - a (D+1)xI data matrix, where D is the data dimensionality
%            and I is the number of training examples. The training
%            examples are in X's columns rather than in the rows, and
%            the first row of X equals ones(1,I),
%        w - a Ix1 vector containing the corresponding world states for
%            each training example,
%        var_prior - scale factor for the prior spherical covariance,
%        X_test - a data matrix containing training examples for which
%                 we need to make predictions,
%        initial_psi - Ix1 vector that represents the start solution,
%        kernel - the kernel function,
%        lambda - the parameter used in the Gaussian kernel.
% Output: predictions - 1xI_test row vector which contains the predicted
%                       class values for the input data in X_test.
%         psi - Ix1 column vector that contains the coefficients for
%               the activation function.
function [predictions, psi] = fit_klogr (X, w, var_prior, X_test, ...
    initial_psi, kernel, lambda)

    % Compute K[X,X].
    I = size(X,2);
    K = zeros(I,I);
    for i=1:I
        for j=1:I
            K(i,j) = kernel(X(:,i), X(:,j), lambda);
        end
    end
```

```
    % Compute K[X,X_test].
    I_test = size(X_test,2);
    K_test = zeros(I, size(X_test,2));
    for i=1:I
        for j=1:I_test
            K_test(i,j) = kernel(X(:,i), X_test(:,j), lambda);
        end
    end

    % Find the MAP estimate of the parameters psi.
    options = optimset('GradObj','on','Hessian','on');
    psi = fminunc(@(psi) fit_klogr_cost(psi, X, w, var_prior, K), ...
                  initial_psi, options);

    % Compute the Hessian at psi.
    H = -diag(repmat(1/var_prior,1,I));
    ys = sigmoid(psi' * K);
    for i=1:I
        H = H - ys(i) * (1-ys(i)) * K(:,i) * K(:,i)';
    end

    % Set the mean and variance of the Laplace approximation.
    mu = psi;
    var = -inv(H);

    % Compute mean and variance of the activation.
    mu_a = mu' * K_test;
    var_a_temp = X * var * K_test;
    var_a = zeros(1,I_test);
    for i=1:I_test
        var_a(i) = X_test(:,i)' * var_a_temp(:,i);
    end

    % Approximate the integral to get the Bernoulli parameter.
    predictions = sqrt(1 + pi / 8 .* var_a);
    predictions = mu_a ./ predictions;
    predictions = sigmoid(predictions);
end
```

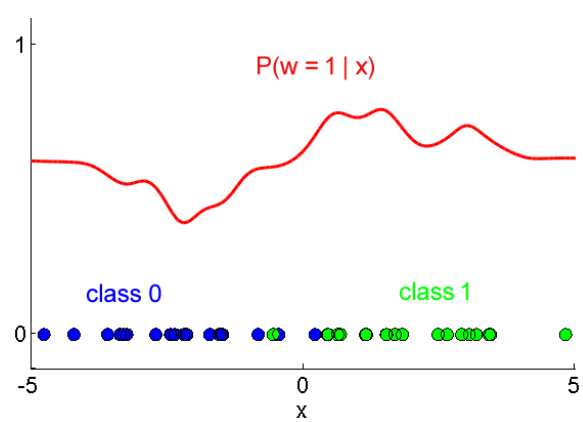


Figure 9.6 Kernel logistic regression model fitted to 1-dimensional dataset.

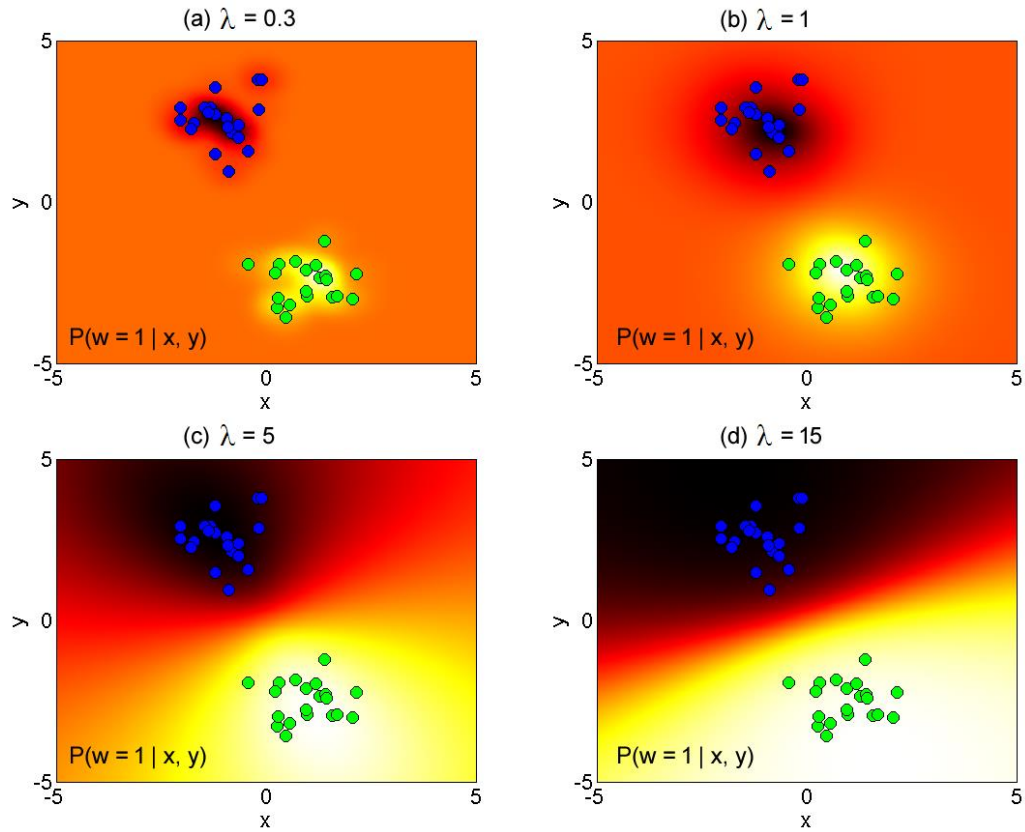


Figure 9.7 Kernel logistic regression model fitted to 2-dimensional dataset. The parameter λ used in the Gaussian kernel is varied while everything else is fixed, (a) $\lambda = 0.3$, (b) $\lambda = 1$, (c) $\lambda = 5$, and (d) $\lambda = 15$. As λ increases, the areas for both classes (0 and 1) spread out, until they collide with each other.

9.6 Incremental fitting for logistic regression

```
% Description: Incremental fitting for logistic regression.
% Input: X - a (D+1)xI data matrix, where D is the data dimensionality
%            and I is the number of training examples. The training
%            examples are in X's columns rather than in the rows, and
%            the first row of X equals ones(1,I),
%        w - a Ix1 vector containing the corresponding world states for
%            each training example,
%        X_test - a data matrix containing training examples for which
%                 we need to make predictions.
% Output: predictions - 1xI_test row vector which contains the predicted
%                       class values for the input data in X_test.
function predictions = fit_inclr (X, w, X_test)
    K = 20;
    phi_0 = 0;
    phi = zeros(K,1);
    I = size(X,2);
    a = zeros(I,1);
    D = size(X,1) - 1;
    xi = zeros(D+1,K);
```

```
    for k = 1 : K
        a = a - phi_0;

        options = optimset('GradObj','on');
        initial_x = ones(2+D+1, 1);
        x = fminunc(@(x) fit_inclr_cost(x, X, w, a), initial_x, options);

        phi_0 = x(1);
        phi(k) = x(2);
        xi(:,k) = x(3:end);

        for i = 1 : I
            f = atan(xi(:,k)' * X(:,i));
            a(i) = a(i) + phi_0 + phi(k) * f;
        end
    end

    I_test = size(X_test,2);
    predictions = zeros(1,I_test);
    for i = 1 : I_test
        act = phi_0;
        for k = 1 : K
            f = atan(xi(:,k)' * X_test(:,i));
            act = act + phi(k) * f;
        end
        predictions(i) = sigmoid(act);
    end
end
```

```
% Description: Cost function for incremental logistic regression.
% Input: x - (2+D+1)x1 column vector subject to minimization,
%        X - a (D+1)xI data matrix, where D is the data dimensionality
%            and I is the number of training examples. The training
%            examples are in X's columns rather than in the rows, and
%            the first row of X equals ones(1,I),
%        w - a Ix1 vector containing the corresponding world states for
%            each training example,
%        a - a Ix1 vector which contains the activations for all the
%            training examples.
% Output: f - the value of the function,
%         g - the gradient.
function [f, g] = fit_inclr_cost (x, X, w, a)
    I = size(X,2);
    D = size(X,1) - 1;
    f = 0;
    g = zeros(2+D+1, 1);
    temp1 = x(3:end)' * X;
    for i = 1 : I
        y = sigmoid(a(i) + x(1) + x(2) * atan(temp1(i)));
        if w(i) == 1
            f = f - log(y);
        else
            f = f - log(1-y);
        end
```

```
        temp2 = y - w(i);
        g(1) = g(1) + temp2;
        g(2) = g(2) + temp2 * atan(temp1(i));
        g(3:end) = g(3:end) + temp2 * x(2) * (1/(1+temp1(i)^2)) * X(:,i);
    end
end
```

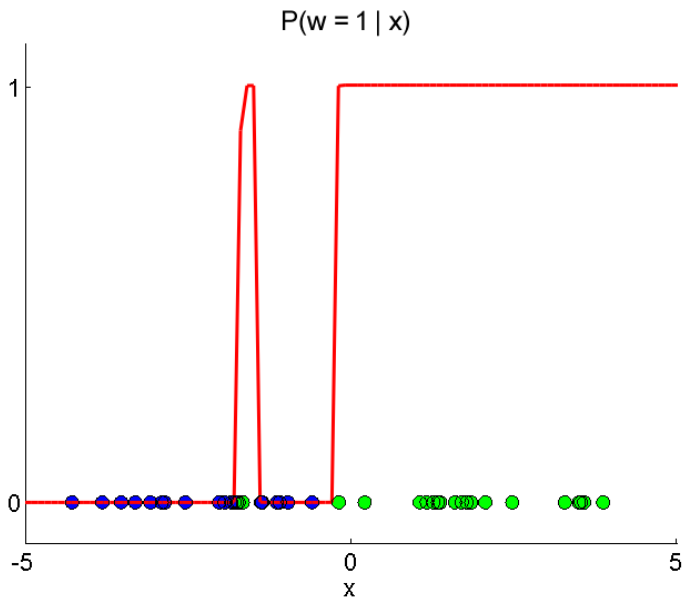


Figure 9.8 Incremental logistic regression model fitted on 1-D data using K = 10 atan functions.

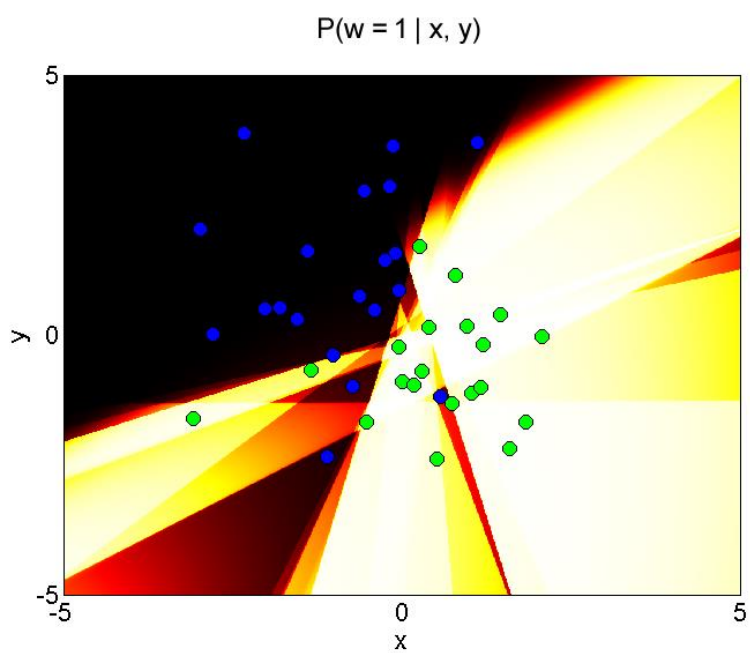


Figure 9.9 Incremental logistic regression model fitted on 2-D data using K = 30 atan functions.

9.7 Logitboost

```
% Description: Logitboost.
% Input: X - a (D+1)xI data matrix, where D is the data dimensionality
%            and I is the number of training examples. The training
%            examples are in X's columns rather than in the rows, and
%            the first row of X equals ones(1,I),
%        w - a Ix1 vector containing the corresponding world states for
%            each training example,
%        X_test - a data matrix containing training examples for which
%                 we need to make predictions,
%        Alpha - (D+1)xM matrix which contains the parameters for the
%                the weak classifiers in its columns.
% Output: predictions - 1xI_test row vector which contains the predicted
%                       class values for the input data in X_test.
function predictions = fit_logitboost (X, w, X_test, Alpha)
    K = 10;
    I = size(X,2);
    a = zeros(I,1);
    phi_0 = 0;
    phi = zeros(K,1);
    c = zeros(K,1);
    M = size(Alpha,2);

    for k = 1 : K
        % Find best weak classifier.
        current_max = -1;
        for m = 1 : M
            value = 0;
            for i = 1 : I
                f = Alpha(:,m)' * X(:,i);
                if(f < 0)
                    f = 0;
                else
                    f = 1;
                end
                value = value + (a(i)-w(i)) * f;
            end
            value = value * value;
            if(value > current_max)
                current_max = value;
                c(k) = m;
            end
        end

        % Remove effect of offset parameters.
        a = a - phi_0;

        % Optimize.
        options = optimset('GradObj','on');
        initial_x = [0; 0];
        x = fminunc(@(x) fit_logitboost_cost(x, X, w, a, Alpha(:,c(k))),
            initial_x, options);
        phi_0 = x(1);
        phi(k) = x(2);
```

```
        % Compute new activation.
        for i = 1 : I
            f = Alpha(:,c(k))' * X(:,i);
            if(f < 0)
                f = 0;
            else
                f = 1;
            end
            a(i) = a(i) + phi_0 + phi(k) * f;
        end
    end

    I_test = size(X_test,2);
    predictions = zeros(1,I_test);
    for i = 1 : I_test
        act = phi_0;
        for k = 1 : K
            f = Alpha(:,c(k))' * X_test(:,i);
            if (f < 0)
                f = 0;
            else
                f = 1;
            end
            act = act + phi(k) * f;
        end
        predictions(i) = sigmoid(act);
    end
end
```

```
% Description: Cost function for logitboost.
% Input: x - (2+D+1)x1 column vector subject to minimization,
%        X - a (D+1)xI data matrix, where D is the data dimensionality
%            and I is the number of training examples. The training
%            examples are in X's columns rather than in the rows, and
%            the first row of X equals ones(1,I),
%        w - a Ix1 vector containing the corresponding world states for
%            each training example,
%        a - a Ix1 vector which contains the activations for all the
%            training examples,
%        alpha - (D+1)x1 vector which contains the parameters for the
%                current best weak classifier.
% Output: f - the value of the function,
%         g - the gradient.
function [f, g] = fit_logitboost_cost (x, X, w, a, alpha)
    I = size(X,2);
    f = 0;
    g = zeros(2,1);
    for i = 1 : I
        temp = alpha' * X(:,i);
        if(temp < 0)
            temp = 0;
        else
            temp = 1;
        end
```

```
        y = sigmoid(a(i) + x(1) + x(2) * temp);
        if w(i) == 1
            f = f - log(y);
        else
            f = f - log(1-y);
        end

        temp2 = y - w(i);
        g(1) = g(1) + temp2;
        g(2) = g(2) + temp2 * temp;
    end
end
```

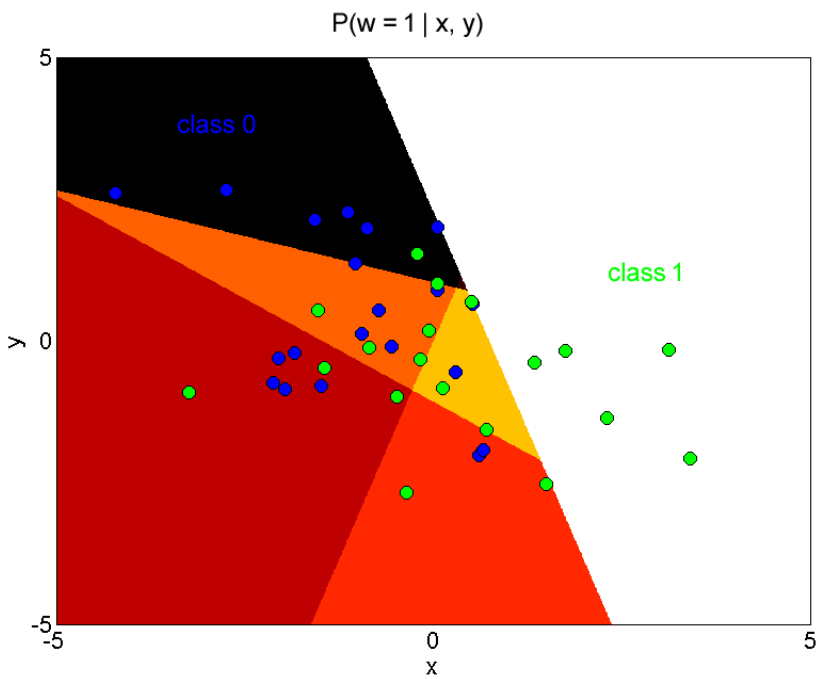


Figure 9.10 Logitboost model fitted on 2-D data using K = 10 step functions.

9.8 Multi-class logistic regression

```
% Description: Multi-class logistic regression.
% Input: X - a (D+1)xI data matrix, where D is the data dimensionality
%            and I is the number of training examples. The training
%            examples are in X's columns rather than in the rows, and
%            the first row of X equals ones(1,I),
%        w - a Ix1 vector containing the corresponding world states for
%            each training example,
%        X_test - a data matrix containing training examples for which
%                 we need to make predictions,
%        num_classes - number of classes.
```

```
% Output: Predictions - (num_classes)xI_test matrix which contains the
%                       predicted class values for the data in X_test.
function Predictions = fit_mclr (X, w, X_test, num_classes)
    % Optimize for phi.
    D1 = size(X,1);
    options = optimset('GradObj','on','Hessian','on');
    initial_phi = ones(D1*num_classes, 1);
    phi = fminunc(@(phi) fit_mclr_cost(phi, X, w, num_classes),...
        initial_phi, options);
    Phi = reshape(phi,D1,num_classes);

    % Predict.
    Phi_X_exp = exp(Phi' * X_test);
    Phi_X_exp_sums = 1 ./ sum(Phi_X_exp,1);
    Predictions = bsxfun(@times, Phi_X_exp, Phi_X_exp_sums);
end
```

```
% Description: Cost function for multi-class logistic regression.
% Input: phi - a (D+1)*num_classesx1 vector that contains the parameters
%              that are subject to optimization,
%        X - a (D+1)xI data matrix, where D is the data dimensionality
%            and I is the number of training examples. The training
%            examples are in X's columns rather than in the rows, and
%            the first row of X equals ones(1,I),
%        w - a Ix1 vector containing the corresponding world states for
%            each training example,
%        num_classes - number of classes.
% Output: f - the value of the function,
%         g - the gradient,
%         H - the Hessian.
function [L, g, H] = fit_mclr_cost (phi, X, w, num_classes)
    % Init.
    L = 0;
    D1 = size(X,1);
    D = D1 - 1;
    Phi = reshape(phi,D1,num_classes);
    num_variables = D1 * num_classes;
    g = zeros(num_variables,1);
    H = [];
    I = size(X,2);
    ddirac = @(x) double(not(x));
    HH = cell(num_classes, num_classes);
    for i = 1 : num_classes
        for j = 1 : num_classes
            HH{i,j} = zeros(D1,D1);
        end
    end

    % Compute the predictions Y for X.
    Phi_X = Phi' * X;
    Phi_X_exp = exp(Phi_X);
    Phi_X_exp_sums = 1 ./ sum(Phi_X_exp,1);
    Y = bsxfun(@times, Phi_X_exp, Phi_X_exp_sums);
```

```
    for i = 1 : I
        % Update log likelihood L.
        L = L - log(Y(w(i),i));

        start = 1;
        for n = 1 : num_classes
            % Update gradient.
            temp1 = (Y(n,i) - ddirac(w(i)-n)) * X(:,i);
            g(start : start+D) = g(start : start+D) + temp1;
            start = start + D1;

            % Update Hessian.
            for m = 1 : num_classes
                temp2 = Y(m,i) * (ddirac(m-n) - Y(n,i)) * X(:,i) * X(:,i)';
                HH{m,n} = HH{m,n} + temp2;
            end
        end
    end

    % Assemble final Hessian.
    for n = 1 : num_classes
        H_n = [];
        for m = 1 : num_classes
            H_n = [H_n HH{n,m}];
        end
        H = [H; H_n];
    end
end
```

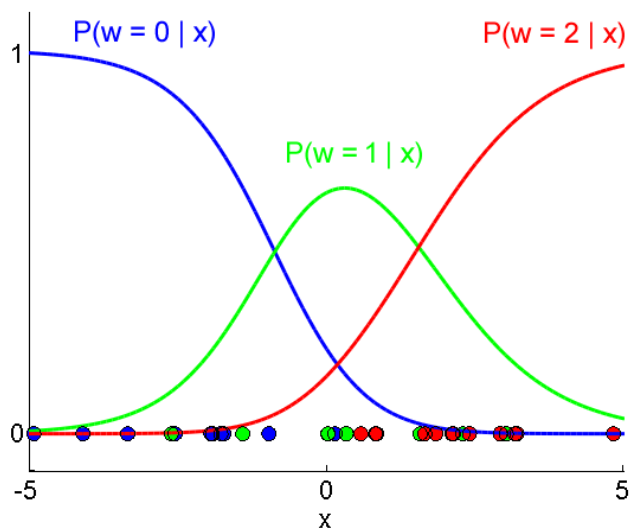


Figure 9.11 Multiclass logistic regression model fitted to 1-D data.

9.9 Multi-class logistic classification tree

```
% Description: Multi-class logistic classification tree.
% Input: X - a (D+1)xI data matrix, where D is the data dimensionality
%            and I is the number of training examples. The training
%            examples are in X's columns rather than in the rows, and
%            the first row of X equals ones(1,I),
%        w - a Ix1 vector containing the corresponding world states for
%            each training example,
%        X_test - a data matrix containing training examples for which
%                 we need to make predictions,
%        J - wanted number of nodes in the tree,
%        G - linear functions in columns used as classifiers,
%        K - number of classes.
% Output: Predictions - (num_classes)xI_test matrix which contains the
%                       predicted class values for the data in X_test.
function Predictions = fit_mclct (X, w, X_test, J, G, K)
    I = size(X,2);
    I_test = size(X_test,2);
    M = size(G,2);
    c = zeros(J,1);
    Lambda = zeros(K,J+1);
    GX = sigmoid(G' * X) > 0.5;
    GX_test = sigmoid(G' * X_test) > 0.5;
    Predictions = zeros(K,I_test);

    % Init queue.
    import java.util.LinkedList;
    queue = LinkedList();
    add(queue, 1:I);

    % For each node in the tree.
    for j = 1 : J
        current_data = remove(queue);
        II = length(current_data);
        l = zeros(M,1);
        for m = 1 : M
            % Count frequency for k-th class in left and right branches.
            n_l = zeros(K,1);
            n_r = zeros(K,1);
            for i = 1 : II
                ii = current_data(i);
                k = w(ii);
                if(GX(m,ii) == 1)
                    n_r(k) = n_r(k) + 1;
                else
                    n_l(k) = n_l(k) + 1;
                end
            end

            % Compute log likelihood.
            sum_l = sum(n_l);
            sum_r = sum(n_r);
            % If this classifier can not separate the data then it should
            % not be applied.
```

```
            if(sum_l == 0 || sum_r == 0)
                l(m) = Inf;
            else
                % Protect against zero values.
                n_l = n_l + 1;
                n_r = n_r + 1;
                sum_l = sum_l + K;
                sum_r = sum_r + K;

                % Compute log likelihood.
                norm_l = log(n_l ./ sum_l);
                norm_r = log(n_r ./ sum_r);
                l(m) = sum(norm_l) + sum(norm_r);
            end
        end
        % Store index of best classifier for this node.
        [~, c(j)] = min(l);
        % Partition in two sets.
        right = GX(c(j),current_data);
        add(queue, current_data(not(right)));
        add(queue, current_data(right));
    end

    % Recover categorical parameters at J+1 leaves.
    for p = 1 : J + 1
        current_data = remove(queue);
        II = length(current_data);
        n = zeros(K,1);
        for i = 1 : II
            ii = current_data(i);
            k = w(ii);
            n(k) = n(k) + 1;
        end
        Lambda(:,p) = n ./ sum(n);
    end

    % Predict by moving each test example from the root to the bottom
    % of the tree, and then taking the corresponding categorical
    % distribution from the leaf node.
    for i = 1 : I_test
        j = 1;
        right = 0;
        tree_level = 0;
        while true
            right = GX_test(c(j),i);
            new_j = 2*j;
            if (right == true)
                new_j = new_j + 1;
            end
            if(new_j > J)
                break;
            else
                j = new_j;
                tree_level = tree_level + 1;
            end
```

```
        end
        node_index_in_last_level_left_to_right = j - 2^tree_level;
        if (right == false)
            leaf_node = node_index_in_last_level_left_to_right * 2 + 1;
        else
            leaf_node = node_index_in_last_level_left_to_right * 2 + 2;
        end
        Predictions(:,i) = Lambda(:,leaf_node);
    end
end
```

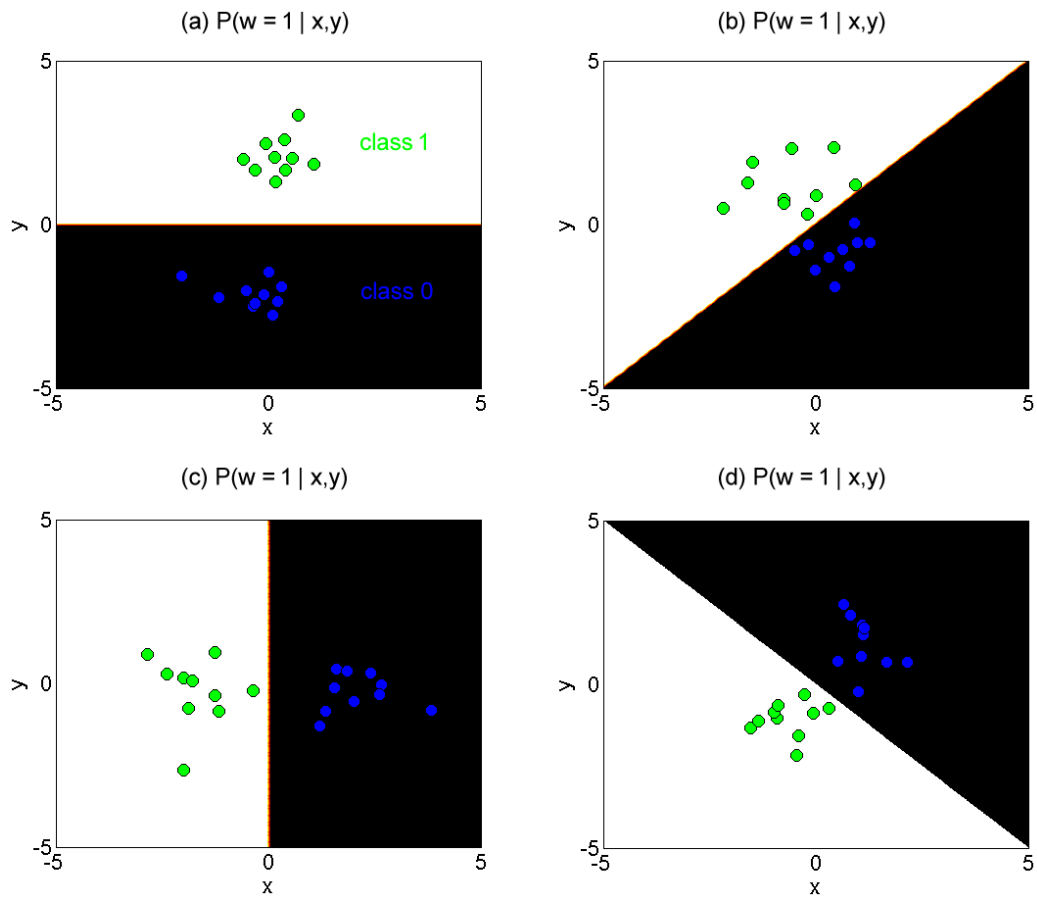


Figure 9.12 Multiclass logistic classification tree. This is the simplest case where the number of nodes in the tree J = 1. In (a-d), for each dataset a different best classifier was chosen from a set of four predetermined classifiers.

Chapter 10

Graphical Models

10.1 Gibbs sampling from discrete undirected model

```
% Description: Gibbs sampling from discrete undirected model.
% Input: Phi - cell of C functions,
%        S - cell of C arrays which represent the domains for the functions
%            in Phi. For example, S{1} = [1 2] means that the function
%            Phi{1} depends on the variables 1 and 2,
%        D - dimensionality of one sample, also the number of
%            variables in the joint distribution,
%        Values - each of the D variables can take K values,
%        T - number of samples to generate.
% Output: Samples - DxT matrix which contains the samples in its columns.
%                   A sample in this case does not contain directly
%                   the values for the variables. Instead, it contains
%                   indices for values in the vector Values.
function Samples = sample_gibbs (Phi, S, D, Values, T)
    Samples = ones(D,T);
    for t = 2 : T
        Samples(:,t) = sample_gibbs_one(Samples(:,t-1), Phi, S, D, Values);
    end
end
```

```
function sample = sample_gibbs_one (sample_start, Phi, S, D, Values)
    K = length(Values);
    C = length(Phi);
    sample = sample_start;
    for d = 1 : D
        lambda = zeros(K,1);
        for k = 1 : K
            sample(d) = k;
            %Compute unnormalized marginal probability.
            lambda(k) = 1;
            for c = 1 : C
                if(ismember(d, S{c}))
                    args = Values(sample(S{c}));
                    lambda(k) = lambda(k) * Phi{c}(args);
                end
            end
        end
        lambda = lambda ./ sum(lambda);
        sample(d) = randsample(1:K,1,true,lambda);
    end
end
```

The example code being small, is also presented here.

```
% Sample.
Phi = {@(x) mvnpdf(x,[0 0],[2 0; 0 2])};
S = {[1 2]};
D = 2;
Values = -5:.5:5;
T = 5000;
Samples = sample_gibbs (Phi, S, D, Values, T);
```

```
discard = 500; % discard first samples
numberOfSamples = (T-discard) / 5; % take every 5-th sample
Samples = Samples(:,discard+1:end);
Samples = Samples(:,1:5:end);

% Plot 2-D histogram.
K = length(Values);
H = zeros(K,K);
I = size(Samples,2);
Samples(2,:) = K + 1 - Samples(2,:);
for i = 1 : I
    col = Samples(1,i);
    row = Samples(2,i);
    H(row,col) = H(row,col) + 1;
end
imagesc(H);
```

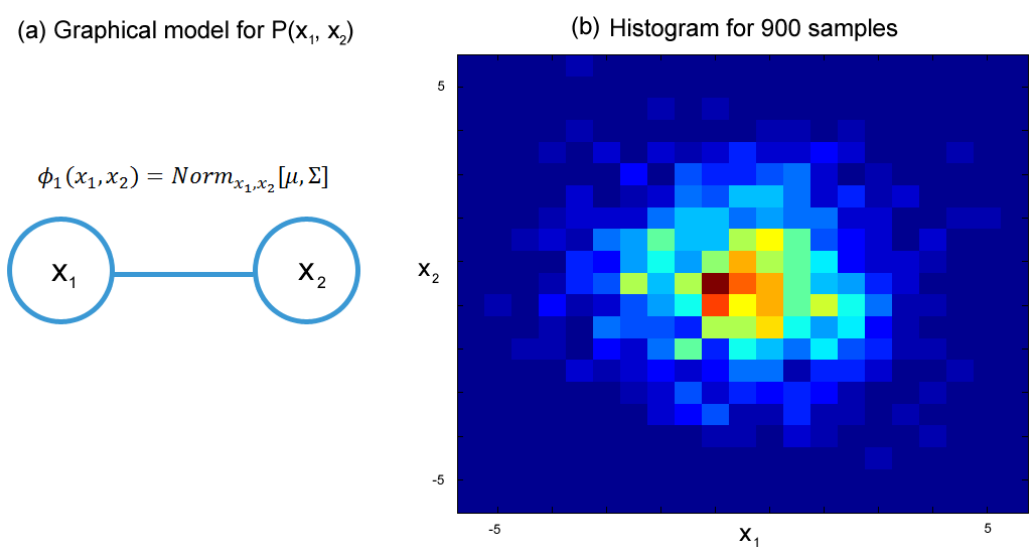


Figure 10.1 Gibbs sampling. (a) The undirected model from which the samples were taken. The parameters used for the potential function ϕ_1 are $\mu = [0\ 0], \Sigma = [2\ 0; 0\ 2]$. Each of the two variables x_1 and x_2 can take values $[-5{:}\,0.5{:}\,5]$. (b) The actual number of samples is T = 5000, but the first 500 samples were discarded so that the initial conditions are forgotten. Then from the remaining samples, every 5th sample was taken, and so $(5000 - 500)/5 = 900$ samples remained in the end.

10.2 Contrastive divergence for learning undirected models

This algorithm is not implemented here as a general function, instead it is presented through an example for a simple graphical model (Figure 10.1 (a)) that has only one potential function. If necessary, this code can be modified to handle complex models as input also.

```
% Generate 2D data from normal distribution.
mu = [4 4];
sig = [1 0; 0 1];
X = mvnrnd(mu, sig, 20)';
```

```
% Create matrix XX that contains indices for the values in X.
% The function sample_gibbs_one uses indices of values, not the
% values directly.
XX = X;
Values = 0:0.5:10; % values that each variable (x and y) can take
value_index_map = containers.Map(Values, 1:length(Values));
for i = 1 : 20
    for j = 1 : 2
        % Get original value from X.
        temp = X(j,i);
        % Snap temp to the closest number in Values.
        m = mod(temp, 0.5);
        fl = floor(temp/0.5);
        if(m > 0.25)
            temp = (fl+1) * 0.5;
        else
            temp = fl * 0.5;
        end
        % Map value to index.
        XX(j,i) = value_index_map(temp);
    end
end

% Create symbolic function phi_1 and its gradient.
syms a b c d;
phi_1_sym = symfun(mvnpdf([a b], [c d], [2 0; 0 2]),[a b c d]);
phi_1_grad = gradient(phi_1_sym, [c d]);

% Learn Theta, the mean of the data.
I = size(X,2);
Samples = XX;
Theta = [0; 0]; % [mu_x; mu_y]
S = {[1 2]};
D = 2;
alpha = 0.1;
iterations = 0;
while true
    % Take one Gibbs sample step from each data point.
    for i = 1 : I
        Phi = {@(x) mvnpdf(x,[Theta(1) Theta(2)],[1 0; 0 1])};
        Samples(:,i) = sample_gibbs_one(XX(:,i), Phi, S, D, Values);
    end

    % Compute the approximate gradient.
    grad = 0;
    mu_x = Theta(1);
    mu_y = Theta(2);
    for i = 1 : I
        current_sample = Values([Samples(1,i) Samples(2,i)])';

        % derivative(log(f(x))) = derivative(f(x)) / f(x) was used to
        % simplify both terms in the approximate gradient.

        f_xi = eval(phi_1_sym(X(1,i), X(2,i), mu_x, mu_y));
        f_xi_star = eval(phi_1_sym(...
```

```
            current_sample(1), current_sample(2), mu_x, mu_y));

        grad_xi = eval(phi_1_grad(X(1,i), X(2,i), mu_x, mu_y));
        grad_xi_star = eval(phi_1_grad(...
            current_sample(1), current_sample(2), mu_x, mu_y));

        temp = grad_xi / f_xi - grad_xi_star / f_xi_star;
        grad = grad + temp;
    end

    % Update Theta with gradient descent.
    Theta = Theta + alpha * grad;

    iterations = iterations + 1;
    if (iterations == 5)
        break;
    end
end

% Plotting code is not shown here...
```

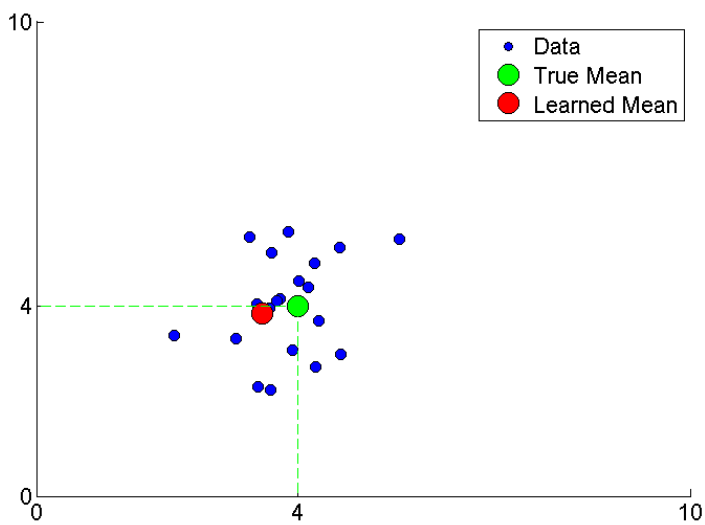


Figure 10.2 Contrastive divergence for learning undirected models (the model used here is shown in (Figure 10.1 (a))). The data was generated from a normal distribution with parameters $\mu = [4, 4], \Sigma = [1\ 0; 0\ 1]$. The learned parameters are $\mu = [\mu_x, \mu_y]$, while Σ was used as in the original distribution. This result was produced after 5 iterations of updating the parameters μ_x and μ_y, with learning rate $\alpha = 0.1$.

Chapter 11

Models for Chains and Trees

11.1 Dynamic programming for chain model

```
% Description: Dynamic programming (Viterbi algorithm) for chain models.
% Input: U - NxK matrix that contains the unary costs,
%        P - 1xN cell of KxK matrices that contain the pairwise costs.
% Output: w - minimum cost path.
function w = viterbi_chain(U, P)
    N = size(U,1);
    K = size(U,2);
    S = zeros(N,K);
    S(1,:) = U(1,:);
    R = zeros(N,K);
    w = zeros(N,1);
    for n = 2 : N
        for k = 1 : K
            temp = S(n-1,:) + P{n}(k,:);
            [m, R(n,k)] = min(temp);
            S(n,k) = U(n,k) + m;
        end
    end
    [~, w(N)] = min(S(N,:));
    for n = N:-1:2
        w(n-1) = R(n,w(n));
    end
end
```

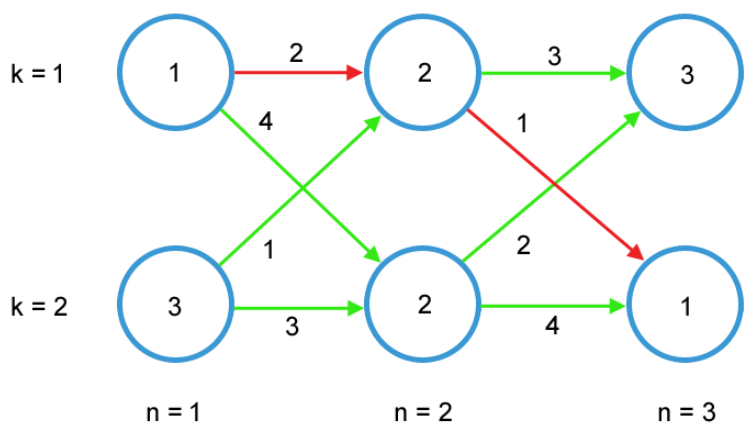


Figure 11.1 Dynamic programming (Viterbi algorithm) example. There are $N = 3$ variables w_n, and each can take $K = 2$ values. The unary costs are inside the nodes, and the pairwise costs are near the arrows. The best path is marked red.

The example code below found the best path [1 1 2] for the graph in (Figure 11.1).

```
U = [1 3; 2 2; 3 1];
P = {[], [2 1; 4 3], [3 2; 1 4]};
w = viterbi_chain(U, P);
disp(w);
```

References

1. Simon J. D. Prince. Computer Vision: Models, Learning and Inference. Cambridge University Press, edition 1 (June 18, 2012).

2. Yi Wang's Tech Notes Blog. URL: http://cxwangyi.wordpress.com/2009/03/18/to-generate-random-numbers-from-a-dirichlet-distribution/. Accessed 11.03.2013.